Blossom Consensus Protocol

*v*2.0.0-pre-release

Devon Tietjen @d-tietjen

Contents

Abstract

This paper describes a novel consensus protocol implementing the systematic sub-sampling of network actors to adopt a shared state in logarithmic time. The protocol exhibits variable Byzantine fault tolerance while allowing for leaderless network participation and fully deterministic behaviors. These features guarantee redundancy as there exists no single point of failure across the network, with members able to participate asynchronously. Our assumptions in developing this protocol suppose that each actor is managed as a replica device with the state of each machine identical to their peers, and given the deterministic nature of the protocol, when presented with equivalent data, each node will reach

a uniform state. Whereas previous protocols are limited to fully serialized behavior, with 'scalable' networks processing transactions at a constant rate and relying on the implementation of external parallelism techniques, the protocol described in this paper introduces parallel behaviors within the core consensus algorithm to allow for scalability without the need for additional tooling. This v2.0.0 pre-release explicitly distinguishes the historical v1 benchmark baseline from the current implementation-aligned v2 protocol, which adds deterministic prefill dispatch, signed reconciliation, and feature-compatible fair ordering. To support these claims, we developed and measured the performance of a prototype network across scales and configurations. The results of this testing displayed a trend towards increased throughput, measured as transactions-per-second, with an increasing number of nodes. We also found that with an increase in minimum resources available to each node, protocol performance increased at a linear rate, until some threshold.

1 Introduction

Extant consensus protocols fail to provide the necessary performance to substantiate their utility, with the limiting factors of these protocols exhibited by high operation costs or limited performance outcomes. While new protocols continue to be developed which are progressively faster than their predecessors, the scalability of said protocols remains non-existent, with performance outcomes remaining constant despite network size. In developing a protocol that displays constant performance, every additional node is a burden on the precursory members. For such protocols, there will always exist an incentive to reduce network participation, which in turn has negative consequences regarding a platform's decentralization. Given these algorithms are primarily designed for substantiating guaranteed network performance in a trustless environment, increased decentralization is considered a necessity. A majority of these protocols present additional risks, many of which have not been well understood prior to their real-world implementation. For the sake of brevity, we will be limiting the examples of such failings to the Bitcoin network and protocol, with the understanding that many other systems exhibit similar failings as a consequence of their consanguineous nature [11]. As discussed in greater detail later in this paper, the failings include the implementation of competitive proofs, the consolidation of resources (compute systems or currency) outside the domain of the decentralized network and the non-deterministic nature of validation proofs.

Working to mediate the aforementioned failings and limitations of existing protocols, this paper presents a novel consensus protocol, `Blossom`, with a variable tolerance for Byzantine (i.e., arbitrary) faults ($f \approx \frac{1}{3}$), deterministic transaction finality and scalable performance with difficulty increases at (non-constant) logarithmic time. The result is a protocol that incentivizes network scaling, introduces supermajority consensus requirements, and mitigates the risks associated with resource accumulation. The most salient risk to our protocol, a Denial-of-Service attack (DoS) was mitigated by removing synchronicity requirements and implementing redundancy measures to guarantee the continuity of consensus despite a node faltering mid-committee. Other risks which we have mitigated in our protocol include Sybil attacks, Routing attacks, and Distributed-Denial-of-Service (DDoS) attacks. In implementing our protocol we recommend participation requirements such as zero-knowledge (ZK) identity verification or network buy-ins. Both of these implementations, as well as incentivizing an increasingly decentralized network, limit the possibility of Sybil attacks; Routing attacks are easily overcome by the encryption of all private data; and DDoS attacks can be limited by isolating excess network activity to each individual node.

To evaluate the validity of these statements we developed a prototype decentralized network (Eden) that implements `Blossom` as its governing consensus protocol. The network is then tested across variable scales and against a variety of scenarios, to provide a reasonable projection of performance. We ran our test across multiple AWS EC2 instances, with applications orchestrated within independent docker containers. We used Kubernetes to manage the orchestration of each docker container within EC2 instances [4]. To benchmark our network we implemented the Prometheus Benchmarking tool to evaluate performance [15].

With these results in mind, this paper outlines the contributions and improvements which have facilitated the aforementioned performance advancements in the following sections: the protocol version boundary [§2], a glossary of terms [§3], the discussion of related work [§4], our assumptions in developing `Blossom` [§5], `Blossom`'s design [§6], the v2 implementation alignment and proofs [§7, §8], the projected performance of `Blossom` [§9]; system benchmarking [§10], theoretical future additions to the protocol [§11], and a conclusion discussing our accomplishments, the implications of said accomplishments, and the prospective applications we believe will provide further improvements to our system [§12].

2 Protocol Version Boundary

This document uses two protocol names deliberately. **Version 1 (v1)** refers to the historical Blossom design and the measured deployment benchmark baseline. **Version 2 (v2)** refers to the implementation-aligned protocol described by the current code, proofs, and projection model. The distinction matters because the paper preserves some v1 language for continuity, while the current trustless protocol claims are made only for v2.

2.1 Version 1: Historical Baseline

Version 1 introduced the core Blossom idea: organize n validators into a deterministic quorum tensor of quorum size q, move block data across $\lceil \log_q n \rceil$ quorum frontiers, and let each validator independently verify the same accumulated epoch state. The measured AWS benchmark figures in this paper are v1 deployment measurements unless a figure explicitly labels a v2 overlay.

The v1 propagation model treated each tensor frontier as a data-spreading consensus wave. When a first-wave quorum failed, older versions mitigated the availability risk by describing a second recursion through the consensus tree. That second recursion is important historically, but it is not the default epoch path claimed for the v2 trustless protocol.

2.2 Version 2: Implementation-Aligned Protocol

Version 2 keeps the deterministic quorum tensor, the $q = 6$ production profile, and the supermajority principle, but changes the data-availability path. It also makes the deployment trust boundary explicit. The code exposes a trusted path for private known-operator clusters and a verified path for trustless operation. The trusted path treats membership as the safety boundary and optimizes for direct dispatch. The verified path signs and verifies the proof-bearing messages that establish public safety.

Before verified consensus begins, v2 performs deterministic prefill dispatch: each node sends its local block to the future tensor contacts that will carry that block through later rounds. This one-hop prefill replaces only the old first data-spreading consensus wave. Later verified rounds still run, and their dispatch, echo, verification, proposal, and commit evidence certifies the canonical epoch state.

The v2 proof boundary also includes pieces that were not fully specified by the historical narrative: signed manifests, runtime reconciliation, round-skip evidence, reconnect admission, feature-code compatibility, mode-gated propagation policies, filtered availability transfer, and fair block ordering under the `sha256+fair-block-ordering` profile. These behaviors are the current trustless claims of the paper. Trusted-mode optimizations are implementation-supported private-cluster optimizations, but they are not substitutes for the verified path's Byzantine-safety assumptions.

2.3 How to Read This Document

When the paper compares v1 and v2, v1 is the baseline and v2 is the current implementation-aligned model. If an older protocol description conflicts with §7, §8, or §13, the v2 implementation-aligned sections are normative. The benchmark section should be read similarly: historical charts are v1 measurements, while blue v2 curves are implementation-aligned overlays that project the v2 path onto the same scenarios for comparison.

3 Terminology

This section will serve to isolate and define select terms or phrases which may be unfamiliar to the reader but are required for the purposes of reviewing this document. Our classification of 'unfamiliar' is limited to the terms or phrases whose official definition is either incorrect or unknown, relative to our use. In providing a description of each term, we expect there exists no other limitation which may deter a reader's understanding of this document.

Epoch A discrete collection of blocks.

Block A discrete collection of transactions.

Transactions A recorded exchange of tokens.

Ledger A record of activity with append-only alterations.

Finality The time it takes for a transaction to be considered irreversible (e.g. locked).

Transaction per Second (TPS) The measure of transactions processed by the network, divided by the finality of those transactions.

4 Related Work

Proof of Work Bitcoin was the first successful implementation of a consensus protocol within a decentralized network infrastructure, implementing the novel Proof-of-Work (PoW) protocol, Bitcoin has provided the philosophical foundation for a majority of all subsequent protocols and networks [11]. The premise of the PoW protocol, separate from the political and economic beliefs embedded within the Bitcoin system, is the belief that a network of pseudonymous and possibly untrustworthy machines can operate in a manner that guarantees the presence of a correct and shared global state. The guarantees of correctness provided by the system are outlined as probabilistic, with a tolerance of $P < 99.99\%$, where P is representative of the probability by which an attacker could "catch up" with the valid network state. This measurement is important as the correct branch of the blockchain is identified by length, with the longest chain perceived as unanimously correct. In contrast to our work on Blossom, as well as the work described in other papers referenced in this section, Bitcoin implements a novel protocol, Proof-of-Work (PoW), which is considered a highly effective yet inefficient protocol. The unique nature of PoW, which has been emulated by subsequent systems, is the competitive nature of the protocol.

Proof of Stake Unlike the other protocols described here, the cited origin for Proof-of-Stake (PoS) is recognized not as part of a Whitepaper or publication, but as an

idea presented by a user within the *bitcointalk forum* [16]. The protocol, universally seen as the predecessor to Bitcoin's PoW protocol, is a respected alternative as computation cost and related energy consumption required to validate network transactions in PoS are vastly reduced from PoW. These reductions are immensely important given the important conversations regarding blockchain-related emissions, and additional scalability concerns. The key point of departure in PoS from PoW is the use of randomized leader selection, where the chance of a validators selection as the consensus leader is directly associated with the size of their staking position (staking is generally understood as some deposit of crypto made by a network member). By requiring individuals to deposit a reasonably significant sum of tokens to advance the network, any malicious action can be made accountable by seizing or diluting the staked assets. However, despite the benefits of PoS, there remain additional security and governance issues resulting from the protocol, with the additional fact that the performance of PoS systems remains lower than is necessary for universal blockchain adoption.

Practical Byzantine Fault Tolerance A highly influential paper describing "a new state-machine replication algorithm," Practical Byzantine Fault Tolerance (PBFT) is a seminal work that laid the foundation for decentralized consensus protocols [6]. We frequently reference this work within this paper, as the work done on PBFT was highly influential in the design of Blossom. A characteristic of PBFT we find to be highly valuable is the ability for all network members to participate in consensus. This cooperative behavior provides a deterministic finality to all consensus updates while allowing for all nodes (replica machines) to receive the content of state updates at the time of consensus. While PBFT is designed as a "practical" algorithm, when implemented as the consensus protocol for blockchain networks, the internode communication between members is highly inefficient with a communication overhead of $O(n^2)$. When considering the need to handle an exponentially increasing number of communications for every additional node, paired with the increased latency of globally decentralized networks, PBFT fails to be effective at scale. Additional factors which have limited the adoption of PBFT to permissioned networks include the inability of the protocol to successfully mitigate Sybil and DDoS attacks.

HoneyBadgerBFT & Tendermint While these two papers describe unique works, both papers propose improvements to the PBFT algorithm, with the goal of optimizing PBFT for use in blockchain systems. Tendermint is generally understood as the most efficient Byzantine fault tolerant (BFT) protocol implementation, with the highest profile implementation of the protocol in the Cosmos network [3] [8]. We are interested in Tendermint as the proven performance and efficiency improvements displayed in the protocol provide a validated benchmark for us to further improve. HoneyBadgerBFT (HBBFT) is another improvement on the PBFT algorithm; however, unlike Tendermint, HBBFT is designed to increase the attack resistance and asynchrony of the protocol [10]. Our interest in HBBFT is the result of its protocol architecture, which adds additional tolerances to the protocol, reducing the consequences of singular machine failures. We have made vast improvements beyond both of these algorithms; however, they existed as important reference material in the design of Blossom.

Snowflake Snowflake is a novel consensus protocol that has been implemented in the Avalanche blockchain [18] [1]. Since the original release of this paper the protocol has seen subsequent improvements; however, for the purpose of our reference, we will be focusing on the original work cited above. To guarantee a shared global state, Snowflake implements a subsampling algorithm with similarities to gossip protocols [1], in that each node polls a random subset of the population and reaches a supermajority consensus among said subset. This process is repeated until all nodes in the network return a shared state. The ingenuity of this protocol is the embedded novelty of relying on randomized subsampling, rather than some traditional framework to guarantee a shared network state. However, despite the improvements in communication overhead, Snowflake performance displays constant time performance. It is, for this reason, they heavily cite the implementation of "layer 2 scaling solutions" as the method by which they can further increase network performance.

Proof of History Another novel protocol, Proof-of-History (PoH) is a highly optimized consensus protocol that utilizes the ordering of transactions in blockchain systems as a proxy for time [21]. This protocol is notable for its use in the Solana blockchain, the founders of whom were also the authors of the original PoH paper. While the means by which Solana guarantees ordering remains questionable, the need to order transactions remains one of importance. In previous systems, (i.e Bitcoin) transaction order is a means of validating a shared state, while PoH embeds additional value from this behavior. We reference PoH due to the protocol's unique behavior, a fact that enticed us into questioning the purpose of transaction ordering in our protocol.

5 Assumptions

In the design of `Blossom`, we maintain certain assumptions regarding the behavior of computer systems, as they relate to the utility of our protocol. These assumptions are both general and specific, relating to both the hardware and software required to implement our system. Operating with these assumptions we are able to ignore some quotidian aspects of our technology, as well as implement technologies that have been previously untested. We have utilized assumptions during both the development and analysis of our protocol, by means which are reasonable and accounted for at each relevant occasion. To clarify which assumptions the authors employed in preparing this document, we will outline our assumptions in the following points:

Byzantine Behavior As previously defined as equivalent to 'arbitrary' behavior, can be expected to occur at some generalized rate in the natural world. While we have tested our protocol in what is equivalent to laboratory conditions, we suppose the typical fault occurrence of one fault for every 10^3 nodes every two weeks.

Protocol Security Security is of utmost importance, and while this paper may not describe the methods developers may implement to mitigate the risks related to `Blossom`, we recognize that such solutions do exist (e.g. ZK-proofs, financial incentives, and or supplementary consensus protocols) which may resolve said risks.

Fault Disincentives Another topic that we do not directly reference in this document, which would also be valuable in the mitigation of possibly byzantine behavior. Such disincentives would limit or remediate the perpetual misbehavior of select nodes while providing other network members the ability to act on said issues. This and other 'social issues' are not pertinent to the design of this protocol, but pertinent to the implementation of this protocol, thus we believe some such system must be implemented along with the public use of `Blossom`.

Share Credentials An important characteristic of decentralized systems, implementations of which have been proven as both effective and efficient across scales. Recognizing the viability of such an architecture, for the purpose of our development timeline, we chose to utilize a centralized database for actions requiring peer lookup. While we do not expect this behavior to continue beyond this initial proof-of-concept, we expect the use of a database should be negligible to the performance of our protocol.

State Determinism Best defined as a belief that two similar machines running the same software can independently reach an identical state, given both machines are provided the same input data. This assumption requires the data provided to said nodes to be identical in every way (i.e. indistinguishable).

6 Protocol

This section preserves the v1 protocol narrative and vocabulary as the conceptual baseline described in §2.1. The v2 implementation alignment section later in the paper is normative where it updates these descriptions, especially for quorum safety thresholds, prefill dispatch, runtime reconciliation, and fair block ordering. The `Blossom` protocol is a comprehensive system including algorithms that manage the receiving, sharing, and finalization of network updates (transactions), as orchestrated and managed by a decentralized body of trustless members. Any update made to the network is guaranteed as valid by a supermajority of all network members, with said members storing a local replica of the global network state, with the expectation that the state is cryptographically signed by a supermajority of all nodes. With this embedded decentralization, the system is highly redundant requiring greater than $\frac{n}{3}$ concurrent faults to occur for the system to pause validation.

The procedures we have embedded within this system are intended to provide cooperative performance, with each additional node providing greater value to the system rather than accounting as losses potential, while the error tolerances of the protocol only allow for guaranteed positive behavior with the expected occurrence of false-negatives (type II errors). This strategy allows us to reduce the size and complexity of the code base by removing redundancies which guarantees admission of no false positives (type I errors). As previously mentioned, similar to other protocols which are expected to operate in a trustless environment, each validator is required to store a replica of the network state. Each validator then references this local state during transaction verification and consensus. Due to this framework, all behavior that we will describe in this section is understood as replicated across all validator instances.

6.1 Structure

`Blossom` can be implemented on machines across the globe, with variability in network bandwidth and node compute potential, thus there is a need to optimize the protocol to mitigate issues associated with said variability. Our optimizations minimize network traffic during data dissemination while allowing for parallelization during transaction verification. The goal of these improvements is to reduce transaction time-to-finality (latency) and improve transaction processing efficiency, thus increasing the potential network throughput (TPS).

The protocol structure, resultant of these technical improvements, incorporates a system of three (3) distinct se-

rialized operations: ⟨1⟩ block formation, ⟨2⟩ block propagation, and ⟨3⟩ transaction validation. These processes can be understood as serialized stages of the protocol, requiring the previous stage to be completed for the next to begin. While tasks within each stage can be optimized with parallelization, these discrete phases have no such parallel potential. The characteristics of each stage can best be understood as the following:

Block Formation With `Blossom` every network member is able to receive transactions and form blocks in parallel. This is possible given the 'leaderless' nature of the protocol, with the system designed to allow independent members to reach a shared state. When a transaction is received by a member the transaction is *pre-validated*. *Pre-Validation* requires the member to check the transaction against the local state and return a preliminary verification of the transaction. Verified transactions are then added to the member's local block, with the block closed once a predetermined cap has been reached (1 MB).

Block Propagation As every network member is in possession of a unique block of transactions, this stage of the protocol deals with the dissemination of said block in the most efficient manner. Our solution involves a recursive process of sub-quorum consensus, with each member participating in a quorum (q) of size $q = 6$. We have selected six (6) for our quorum sizes as it is the largest value which allows for both a supermajority (s) to be found ($s = \frac{2}{3}$) while presenting a fault-tolerance (f) greater than one (1). Each quorum undergoes a procedure of message passing, with each message furthering consensus among all quorum members. The distinct message types are: *dispatch*, *echo*, *verification*, *proposal*, and *commit*. Once a quorum is completed — each member has received the unique block(s) from their peers and agreed on a shared outcome — the member moves into the next quorum where the process is repeated. The membership and order of each quorum is deterministically computed before block propagation begins, to guarantee a shared outcome. Once a member has iterated through every quorum, they should now possess the collective data of every member on the system.

Transaction Validation To process the newly collected transactions in a concurrent manner we implement an ordered data structure (B+ tree) for the storage of blocks, with transactions stored in a *B+ tree* within each block. Given the serialized nature of transaction validation, we have worked to implement some parallelization within this procedure. Acknowledging that signature verification is the process with the greatest cost ($\sim 100,000$ compute cycles per signature), we looked to implement parallelization for this computation. Given the serialized process of updating the ledger is less costly per transaction ($\sim$ 100 compute cycles per signature), every parallel thread verifying signature correctness drastically reduces *transaction validation* latency.

The cumulative outcome resulting from successfully processing these three (3) stages is representative of a full protocol cycle, with the outcome of said cycle validating the state unification of a decentralized and trustless body. The colloquial terminology which numerically represents this protocol cycle is understood as the protocol *Block Time* (B_t).* The stages described above will be discussed in further detail in subsequent sub-sections in this document with additional nuance and descriptions [§6.3, §6.4, §6.5 respectively].

6.2 Initialization

Before the blockchain is an active and autonomous system it must first be initialized, and in the case of a network, outage reinitialized. The procedure for primary initialization, colloquially acknowledged as "genesis," is generally undertaken by a platform's founding members. We do not intend to stray from this norm, defining a predetermined "genesis state" with platform insiders representing the initial network members. Depending on the context of the protocol's implementation additional members may or may not be added.

To guarantee the immediate operation of the network following *genesis* the platform is expected to maintain a network size greater than or equal to the minimum quorum size of the protocol implementation ($n \geq q$). While for the purposes of this paper, we have implemented a quorum size of six (6) ($q = 6$), others may adopt differing quorum characteristics. However, despite any alteration in quorum specifications, the prior rules will still apply. Once a consensus has been reached among inceptive nodes and the genesis state has been finalized, the recursive state validation of the protocol shall continue indefinitely. This operation of *initialization* is expected to occur once, at the inception (genesis) of the network implementing this protocol.

Reinitialization is implemented in the case that the protocol is forced to 'stall' due to the failure of nodes to reach a shared state. A 'stalled' network is one where, while some sub-quorums may successfully process blocks, epochs fail to be finalized by the necessary number of signatures to be considered 'globally validated.' The procedure for network *reinitialization* is different from *initialization* and requires the following assumptions to be true:

Block Time is the measure of time it takes for network members to create, share, and validate their block amongst the global network.

- A viable number of active nodes exists, with the expectation that the proportion of active nodes to Byzantine nodes is within a reasonable margin.
- All nodes store a replica of the global ledger, which dates back a minimum number of epochs.
- At some point in this shared record there was a state which can be agreed upon by a supermajority of nodes.

With these assumptions in place, *reinitialization* allows participating nodes to undergo a new process of consensus, where some supermajority of nodes can agree to a shared state by adopting a previously validated state, then restart the regular operation of the network. If the need arises for *reinitialization*, we expect a supermajority consensus to require only a minimal degree of transposition. Reducing transposition is important because every epoch traversed into the past, and the data contained within those epochs, will be invalidated.

6.3 Communication Model

The topology required to facilitate `Blossom` is maintained under the purview of a fully decentralized network architecture, where every node in the system is able to communicate with any other node in the system without obstruction. However, during the act of consensus, to limit any obtrusive and sub-optimal transmissions, the network arranges itself into a highly structured distributed network. Restated, this 'structured' composition of the network is a temporal arrangement exclusive to the operation of consensus, with all non-consensus behavior handled as direct peer-to-peer (`P2P`) communication.

The clear and delineated distinction between structured and unstructured network communication is important to understand, as both are utilized for specialized operations. *Unstructured communication* refers to non-consensus exchanges between two nodes, such as data queries, transaction requests, etc. The goal of such *unstructured communication* is to prioritize accessibility while reducing network latency on jobs that do not require additional validation throughout the network. *Structured communication* relates to the transmission of data relevant to consensus and is implemented to optimize data transmission to maximize network throughput. This is achieved by limiting duplicate data transmissions and limiting the number of communications required to reach consensus while allotting standard tolerances across replica nodes. For the purposes of this paper, we will be limiting all future discussions regarding the behavior of communication to the purview of *structured communication*.

6.4 Consensus Tree

To facilitate the structured communication of data across the network, `Blossom` adopts algorithms that compose the existing network mesh into a deterministic pattern, to guarantee all network members are able to independently reach a uniform topology (to be referred to as the 'consensus tree'). As mentioned in the previous section, this network structure is designed as a highly optimized scheme with the intention of reducing latency, mitigating Byzantine faults, and allowing for exact consistency across replicas. While certain concurrent behaviors, such as state replication, are required for the successful implementation of the *consensus tree*, we can assume nodes successfully operate as independent replicas; otherwise, the use of a consensus protocol would be trivial. Our description of the network topology as a consensus tree is didactic, as when the network is organized in the deterministic pattern previously referenced, the distribution of network nodes and their relationships is visually representative of a tree (see *Figure 1*).

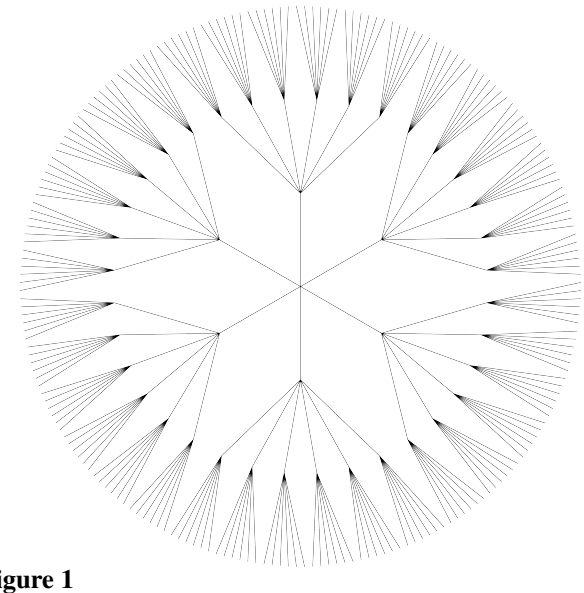


Figure 1

The consensus tree visualized, as the parallel quorums which occur for a singular node to reach consensus. This diagram presents a network size of 216 ($n = 216$), and quorum size of 6 nodes ($q = 6$).

The consensus tree structure represented in *Figure 1* is made up of an exterior perimeter, implied by the outermost branches of the 'tree,' with the number of branches that meet the perimeter equivalent to the total number of nodes on the network ($n = 216$). Within the tree there then exist moments of union where many branches merge. These moments of intersection represent the occurrence of a quorum, where q nodes come together to achieve a unified state. In *Figure 1* there are three (3) distinct union hierarchies, a value which can be derived from the logarithm of all network members $\lceil \log_q n \rceil$. The centroid of the diagram, represented as the moment where all independent branches reach a shared union, is

representative of all network members achieving consensus.

Figure 1 should be read as the historical v1 consensus-tree visualization. The v2 implementation-aligned topology is better viewed as a tensor schedule, where each node can independently compute the future quorums it will meet and can prefill those future contacts before the first verified quorum round. This alternative view is shown in *Figure 2*. The prefill stage is an availability push only; the later dispatch, echo, verification, proposal, and commit stages remain the source of signed consensus evidence.

To further elaborate on the mechanics of the consensus tree: the data structure is a deterministic algorithm that allows each node to concurrently determine the number of quorums required to reach a shared state (i.e. consensus), as well as determine the specific nodes within each quorum. Given some arbitrary number of nodes (n) and assuming a quorum size of six (6) ($q = 6$), we can estimate the number of quorums necessary to reach consensus as $\log_q n$. We subdivide the network into quorums, with each node participating in a minimum of one (1) quorum, as a means of simulating global P2P communication without the need for true P2P communication. In assuming correct behavior across all nodes, in a classical P2P system, the number of messages required to achieve consensus is equivalent to $M_{\text{msg}}(n^2 - n)$, where M_{msg} is the minimum per-peer message factor needed to achieve consensus. Given an increase in network size induces an exponential increase in network load, these protocols are unable to scale, and thus untenable for a decentralized network — such as the issue for PBFT and other similar BFT-based protocols. In subdividing a network into quorums of limited size, such as those of six (6) nodes, our message overhead is reduced to $M_{\text{msg}}(q^2 - q) \times \lceil \log_q n \rceil$. Restructuring the network in this manner the communication cost for each node is reduced from $O(n^2)$ to $O(\log n)$. The efficiency of this communication design, in combination with our parallelized network model, allows us to offer immense gains in network throughput when compared to previous systems.

The mechanics which allow us to provide the aforementioned efficiencies are best exemplified in *Figure 3*, which uses the same tensor-coordinate vocabulary as *Figure 2*. In the v2 view, the selected node first performs quorum 0 prefill dispatch to the future tensor contacts that will carry its block. This is not a certificate. It is a deterministic availability push that prevents the local block from being trapped at the leaf if the old first data-spreading round fails. The later verified path then proceeds with the remaining tensor dimensions only. For $q = 6$ and $n = 36$, $D(n, q) = 2$, so v2 performs one prefill hop and one remaining verified quorum round rather than two verified data-spreading rounds. That remaining verified round still runs dispatch, echo, verification, proposal, and commit, and therefore remains the source of signed consensus evidence. If there is redundancy in the system, it is due to a non-optimal number of network nodes ($n \neq 6^{\mathbb{Z}}$).

The details of data dissemination will be discussed in full in a future section (*block propagation* [§6.6]); however, for sake of clarity the following description will summarize the historical two-round union scheme that v2 compresses. When a node enters a quorum they may or may not be in possession of some unique dataset ($\mathcal{D}$) which may be relevant to their peers. In the case they do possess such data, they will share their dataset in full with their peers and said peers will do the same. Each member will then merge their received datasets via a union operation to achieve a shared outcome. Given a matrix $X = (x_{ij})_{1 \leq i,j \leq 6}$ and some quorum (q_i) on the i-th row during round one (1), and some quorum (q_j) on the j-th column during round two (2), the historical operation is represented as the following:

$$\Sigma_i = \bigcup_{\alpha=1}^{q} \mathcal{D}_\alpha^i, \qquad \Sigma_j = \bigcup_{\beta=1}^{q} \left(\mathcal{D}_\beta^j \cup \Sigma_i \right).$$

As mentioned previously, the outcome of the above operation should see all participating nodes reach a shared state, the equality of which can be validated retrospectively. To verify concurrent data accumulation across members, each node will maintain a local `btreemap`, guaranteeing a consistent ordering of data across nodes while efficiently removing any duplicate data entries.

While the above method is applicable in a non-adversarial (i.e. trusted) environment, the trustless path requires explicit evidence thresholds. The current implementation uses a supermajority threshold of $S(q) = q - \lfloor q/3 \rfloor$. For $q = 6$, a stage may make liveness progress without two slow or absent members because $S(6) = 4$. The stronger safety statement for conflicting supermajority proofs uses the honest-overlap bound $F(q) = \lfloor (q - 1)/3 \rfloor$; for $q = 6$, this admits one Byzantine equivocator inside a quorum. Earlier versions described a full second recursion of the consensus tree as the primary mitigation for failed first-round dissemination. In v2, that mitigation is replaced by deterministic prefill dispatch before verified consensus and by signed reconciliation when summaries are unavailable.

The historical recursion is better described as an expansion over a frontier of current holders, rather than as a quorum applied to a single quorum symbol. Let $\mathcal{H}_0(i) = \{i\}$ be the initial holder set for node i's local block. For a round that varies tensor dimension r_t, define the next holder frontier as:

$$\mathcal{H}_{t+1}(i) = \bigcup_{u \in \mathcal{H}_t(i)} Q_u(r_t), \qquad r_t = t \bmod D.$$

For a two-dimensional historical tree, one complete pass followed by the legacy second recursion can then be written as:

$$\mathcal{H}_0 \overset{r=0}{\Rightarrow} \mathcal{H}_1 \overset{r=1}{\Rightarrow} \mathcal{H}_2 \overset{r=0}{\Rightarrow} \mathcal{H}_3 \overset{r=1}{\Rightarrow} \mathcal{H}_4.$$

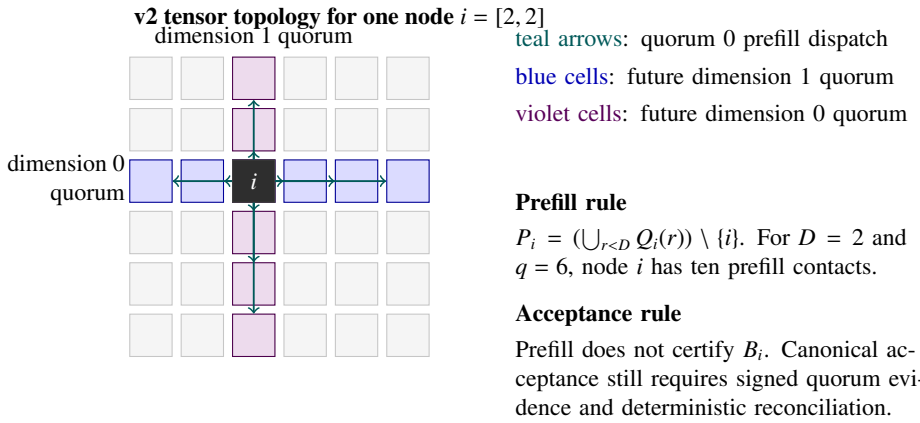


Figure 2

A v2 topology option for the same quorum model. Instead of depicting a radial tree, the diagram shows one node's deterministic tensor coordinates and its non-consensus quorum 0 prefill contacts. Prefill improves initial availability, but canonical acceptance still comes from signed quorum evidence and deterministic reconciliation.

This is the precise form of the older shorthand $q_{i_1} \Rightarrow q_{j_1}(q_{i_1}) \Rightarrow q_{i_2}(q_{j_1}) \Rightarrow q_{j_2}(q_{i_2})$: each step expands every current holder through the quorum for the next tensor dimension. The historical goal of that sequence was to preserve supermajority agreement when each threshold domain remains below the quorum Byzantine fault bound $F(q) = \lfloor (q-1)/3 \rfloor$. The v2 path keeps the same supermajority principle, but reaches the durability target through prefill dispatch, manifest evidence, and reconciliation rather than by making a full second recursion the default epoch path.

Referencing *Figure 3*, the coordinate-line sets from the perspective of node $[0, 0]$ in a 6×6 tensor are as follows. V2 prefill contacts both sets before verified consensus begins; the default path then skips the old first verified data wave and enters the remaining verified dimension:

$$\begin{aligned} Q_{[0,0]}(1) &= \{[0, c] \mid c \in \{0, \ldots, 5\}\} \\ &= \{[0,0], [0,1], [0,2], \\ &\quad [0,3], [0,4], [0,5]\}, \end{aligned}$$

$$\begin{aligned} Q_{[0,0]}(0) &= \{[c, 0] \mid c \in \{0, \ldots, 5\}\} \\ &= \{[0,0], [1,0], [2,0], \\ &\quad [3,0], [4,0], [5,0]\}. \end{aligned}$$

The novel algorithms we implement to yield a useful *consensus tree* include our *member shuffling algorithm* (MSA) and quorum selection algorithm (QSA), with the compute cost of said algorithms being $O(n)$ and $O(\log n)$ respectively, where n is representative of the network size (i.e. number of network nodes). The MSA, while important for `Blossom` implementations on public networks with non-restrictive communication, is a relatively expensive procedure inefficient for protocol implementations on private networks.

To construct the *consensus tree*, the QSA derives a non-biased assembly with the ability for said assembly to be 'biased' (i.e. randomized) by implementing the MSA. To efficiently randomize the data structure, the MSA implements the *Durstenfeld Shuffle Algorithm* [7]. This randomization is made deterministic by the implementation of a cryptographic seed, with the seed in question queried from the hash of the previous epoch — the process of computing the *consensus tree* is expected to occur once each epoch, thus the use of the previous epoch for seeding this computation is both highly secure (the epoch hash derives 2^{256} unique variations) and serves as a secondary concurrency guarantee (nodes with variable states should construct differing *consensus tree's* thus appear faulty to their non-faulty quorum members).

6.5 The Ledger

The 'ledger' is a globally consistent record of platform-specific data that facilitates the accounting of transactions and tracking of network assets in a distributed manner. Every node on the network is expected to maintain a replica of the global ledger, with updates validated by a supermajority of network participants — with said endorsements guaranteed by the use of cryptographic signatures. The ledger is structured in the form of an append-only sequential database, colloquially known as a "blockchain," with the goal of ordering write requests atomically without collisions. To properly manage sequential write requests the ledger is designed as a NoSQL database, managing memory by implementing a data structure similar to an LSM Tree [20][13].

The novelty of our ledger implementation is a result of our unique consensus protocol, allowing for blocks to be formed in parallel. To guarantee a consistent atomic state, despite the presence of parallel blocks, ledger additions are organized by

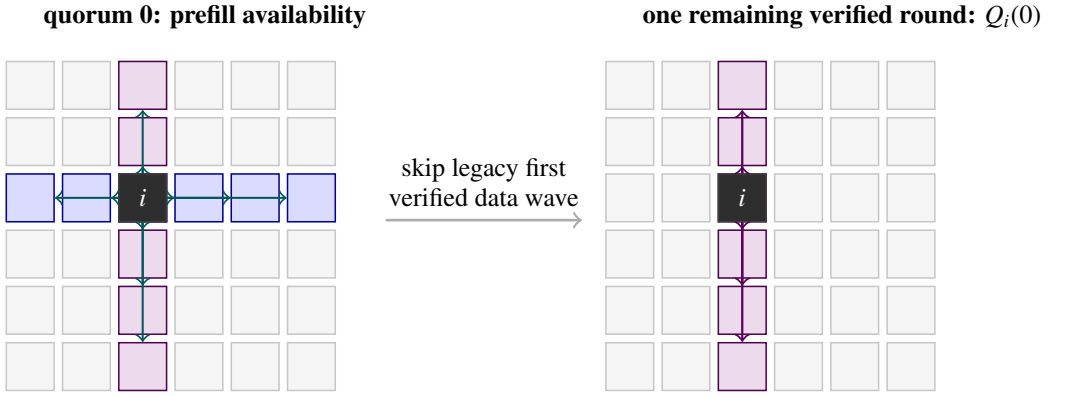


Figure 3

V2 quorum stages for a selected node $i = [2, 2]$ in a 6×6 tensor. Quorum 0 prefill dispatch pushes the local block to future row and column contacts, replacing the old first verified data-spreading wave. The two-dimensional example therefore has one remaining verified quorum round, which still provides signed consensus evidence.

major and minor data structures. Minor data structures refer to individual blocks which store some number of transactions, while major data structures refer to individual epochs which store some uncapped number of blocks (loosely associated with the number of participating nodes). The goal of this structure is to serialize transactions for concurrent processing by independent actors.

For our specific ledger implementation the term "blockchain" is imprecise as the blocks we manage, while sequentially ordered, are independent of their fellow blocks. To elaborate on our unique architecture from preceding systems we can look to the Bitcoin protocol; within the Bitcoin protocol, every consecutive block has embedded within its header the hash of the previous block (see *Figure 4*) [11]. For `Blossom` every block embeds some sequential data within its header; however, rather than embedding the hash of the previous block we embed the hash of the previous epoch. This difference is important as the order of blocks within each epoch is reliant on the alphabetization of said block's hash, rather than a time-stamping mechanism. To verify transaction order, block order, and epoch order, we use novel mechanisms which will be discussed in the following sections (see *Block Formation* [§6.6], *Block Propagation* [§6.7], and *Transaction Validation* [§6.8]).

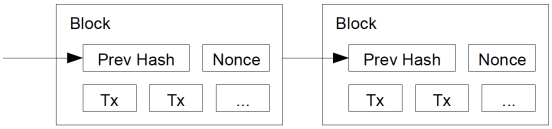


Figure 4

Bitcoin's method for adding a timestamp to blocks, by inserting the hash of the previous block within the header of the consecutive block [11].

6.6 Block Formation

When an outside party (the "client") sends a transaction to the network, said transaction is routed to one of the many validator nodes on the network, with the expectation that the validator will distribute the transaction to their peers. Prior to the distribution of the transaction, it is *pre-validated* by the node; this process preliminarily verifies the validity of a transaction to provide an immediate response to the client. This response is non-binding as the transaction in question may not be included in the final ledger; however, within a reasonable margin of certainty, the client can move forward with the expectation that the transaction will be finalized as

valid by the network.

To provide a preliminary transaction verification, the node in question who receives the transaction references their local ledger, and given the contents of the transaction hold up to scrutiny, the transaction is *pre-validated*. The qualities of the transaction which are scrutinized during this process include verification that the public keys of both the transaction sender and transaction receiver are present in the ledger, that both individuals maintain the proper quantity or class of assets required to facilitate the transaction, the epoch nonce of the transaction is correct, and the cryptographic signatures which validate the individuals as signatories of the transaction are valid.

Following approval, the transaction is then added to a LinkedHashMap (the 'transaction map' or 'LHM') data structure (e.g. data structure which maintains insertion order) maintained within the node's local *build block* [14]. The build block is representative of the mutable state between block initialization and closure, where transactions may still be added to said block. Each block maintains a max size, equivalent to $\sim 1,000$ transactions or $\sim 200MB$, and once this threshold is reached the block is closed. To verify the closure of a block, the block's contents (e.g. transactions) are hashed, with the hash signed by the validator. This new hash (the 'block hash'), signature, and the node's public key are then stored in the header of the block. To compute the 'block hash' `Blossom` implements a procedure similar to that outlined in the Bitcoin whitepaper (see *Figure 4*) [11]. To summarize, each transaction in the block computes its unique hash, the result of which is concatenated with the hash of the previous transaction. This process is repeated sequentially across the transaction map, with the resulting hash representative of the entire block. The 'block hash' is both a unique identifier representing each discrete dataset (block or epoch) and a process that is easily repeated by auditors. To verify the legitimacy of any dataset, every transaction, block, and epoch is expected to maintain a cryptographic signature. Given the hash, public key, and signature are all present in the header of that dataset, any actor can validate the legitimacy of the stored information.

Figure 5 visualizes the previously described process of cumulatively concatenating transaction hashes to generate an overarching block hash. We considered this process to be generalized, as it is able to tokenize and sequence any possible transaction type while guaranteeing the authenticity of each transaction using embedded cryptographic signatures. If additional specificity is required by auditors, such as revealing the unique hash generated by the validator at each transaction instance, a separate sequential data structure, such as a vector, may be stored within each block. This new data structure would maintain the same sequencing as the original *transaction map* with the difference that at each index position rather than displaying the unique transaction

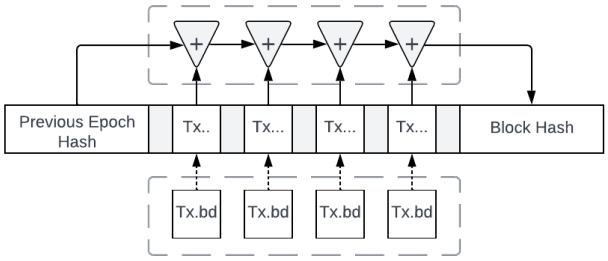


Figure 5

The procedure of generating transaction hashes ('Tx..') from the body content of each transaction ('Tx.bd'), concatenated with a previous hash ('+') to generate a block hash.

hash, the data structure would store the concatenated transaction hash. This vector could then be signed separately or with the other block data. Once signed, a block is perceived as closed and ready to be distributed across the network. This is valuable from a network perspective as this closure mechanism asynchronously signifies the moment at which a block is ready to be distributed. Without some central arbiter or timing mechanism it would be difficult to classify the sequence of block distribution, and by handling this behavior asynchronously, we can distribute blocks in a more efficient manner. However, this true asynchronous behavior is only maintained from the initial distribution of a node's block, as some degree of synchronicity is required during *block propagation*, which will be discussed in detail in the following section [§6.7]. In summary, depending on the outcome of the *consensus tree*, the node will be provided a list of quorum peers. The node will then distribute the blocks it manages locally with its quorums, with the goal that all non-Byzantine nodes accumulate the collection of blocks formed in parallel across the network.

6.7 Block Propagation

Block propagation is the method by which a *consensus tree* is implemented to efficiently distribute independent blocks amongst network members, to quickly reach a shared state (i.e. consensus). As per the mechanics of the *consensus tree*, a subset of nodes (i.e., a quorum) is selected, whose members communicate to reach a shared state. This communication is undertaken with each node sending standardized messages in a serialized order, the fulfillment of which verifies the completion of their communication. The messages in question, in the order they are expected to occur, are: *Dispatch*, *Echo*, *Verification*, *Proposal*, and *Commit*. These message types each play an important role in guaranteeing consistency across independent machines, while also introducing a degree of redundancy to mitigate faults. The characteristics of each message are expected to be derived from

the content of the previous message(s), with this trait in line with our assumptions regarding *state determinism*.

Version 2 adds a deterministic prefill stage before the first verified quorum round. We also call this stage *quorum 0*; the name is only a scheduling convenience. Quorum 0 is a non-consensus availability push in which a node sends its local block to the peers it will meet across later tensor dimensions. It does not replace dispatch, echo, verification, proposal, or commit evidence, and it does not certify a block by receipt alone. Its purpose is to prevent a first round failure from orphaning a local block at the root of the dissemination tree. The later verified quorum rounds still decide which block commitments are accepted into the canonical epoch state.

When a message is distributed, it is the responsibility of each receiving actor to verify the content of the message. This is a consequence of the protocol's decentralized nature, with any node able to communicate with any other node. However, given the computational overhead of each node increases linearly with the size of the network $\langle O(n) \rangle$, we must limit the compute spend on incorrect or invalid transactions. To optimize the protocol, each message implements a header-body format, with universal message content stored in the header and message-unique content stored in the body. By standardizing this format across message types, as well as requiring each header to store: the sender's public key, the last epoch hash, the current epoch nonce, the current height (also known as the 'round') of the consensus tree, and the message signature, if any of these data types are found to be inaccurate when a message is received, the message will be ignored without needing to process the body of that message.

When message headers are verified affirmatively, the message will then be managed as is deemed appropriate given its typology. To discuss each message in the context of their independent qualities, as well as their role within the greater network, they will be described as follows:

Dispatch A *dispatch* is a means by which each actor may distribute any accumulated ledger amendments (e.g. block(s)) with their quorum peers. This block data will be shared with the necessary signatures, to verify the message is accurate while maintaining that each *dispatch* is but an appendant to previous rounds of consensus. To dispense signatures properly, while optimizing for the high compute cost of signature verification, we implement a novel data structure termed the *signature tree*, reducing the time complexity of block verification from $O(n^2)$ to $O(n \log n)$.

Echo Within the classification of an echo message there exist subtypes for given scenarios, with those types being: *Echo Response*, *Echo Request*, and *Echo Redispatch*. An *echo response* is a means of signaling to the quorum that a node received a *dispatch* before doing any work to validate the content of the dispatch. In optimal scenarios, this is the only *echo* type shared between nodes. The latter *echo* types are required given non-optimal behavior, such as when a node receives an *echo* response from a peer, but the node has not yet received the *dispatch* message associated with the *echo response*. Recognizing that they may be missing this crucial data, the node will query the *echo* sender with an *echo request*, to receive the original unadulterated *dispatch* message from the peer. The peer is then expected to return an *echo redispatch*, sharing the missing message with the relevant node. We will discuss the additional benefits of *echo* messages later in this section.

Verification A *verification* shares which block(s) a node believes to be valid or invalid from those they received via *dispatch*, as well as from the block(s) already in their possession. To validate the legitimacy of a block, the block header is referenced to determine whether a block's hash is unique, whether the block has the proper state credentials (epoch nonce and epoch hash), and whether the signatures attributed to the block are valid. The resulting outcome of this validation process is then hashed and signed with the outcome shared within the body of the *verification* message.

Proposal A proposal is the accumulated outcome of *verification* messages, as perceived by each node, the result of which should display an affirmative or negating consensus. An affirmative consensus would occur in the scenario that a supermajority of quorum peers (e.g. $\geq \frac{2q}{3}$) share an identical *verification* message, in that the block(s) determined valid and or invalid are identical. A negating consensus would occur when an affirmative consensus is impossible to occur (e.g. $> \frac{q}{3}$), such as due to a supermajority disagreement. Whatever the result, a *proposal* is a means of compressing the relevant *verification* messages into a concise and verifiable outcome of either `true` or `false`.

Commit Similar to the *proposal*, a commit is the accumulated outcome of *proposal* messages, as perceived by each node. However, unlike the *proposal* message which is used to share the perceived outcome of each node, a *commit* is a definite statement from each node affirming their future behavior, whether it is to operate under the expectation of a `true` or `false` consensus. `True` would signify a consensus was reached during the quorum and some collection of blocks is expected to be ratified. `False` would signify a consensus was not reached and all novel content from the quorum should be ignored.

The outcome of each quorum is dependent on the type of nodes present in each quorum, as well as the context within

said quorum comes together. If enough Byzantine nodes are present within a quorum there will be no outcome, despite any consensus between non-Byzantine nodes, that will be committed as true. A similar non-consensus can occur if the nodes that are placed together within a quorum are less performant than the greater network, the consequences of which will have said nodes forced to prematurely disperse. These behaviors, while non-optimal for any non-Byzantine and performant nodes within each quorum, provide a means of limiting the consequences of any possible faults while allowing systems to fail gracefully in a performant manner. It is here where we must discuss the consequences of faults that occur before or during each quorum, and the methods we adopt to mitigate said faults within the protocol.

As a consequence of this, *cooperative consensus* model there exist two categories of faults that are the predominant complications for this style of the protocol. The first is a consequence of the need for synchronicity (termed "synchronicity failures"), and the second is a consequence of the distributed nature of the need for block accumulation (termed "distribution failures"). To describe each failing appropriately we will detail the variations in the following sections (*Synchronicity Failures* [§6.7.1] and *Distribution Failures* [§6.7.2]).

The measures described in the above sections are meant to outline the solutions we implemented to solve the most common of possible faults which may occur during `Blossom`. These behaviors do not dissuade our belief that other types of faults may occur, and we expect that as additional testing is undergone such issues may arise, but at the same time, we expect that `Blossom's` unique scalability incentivizes a widely decentralized network of non-Byzantine nodes, the existence of which substantiates a lively and performant network, which far exceeds the present outcomes of other precedent systems.

Following the successful procedure of block propagation the node should be party to the aggregated dataset (e.g. blocks) accumulated from each distributed non-Byzantine node on the network. This new dataset (otherwise referred to as the 'unverified state' or 'preliminary epoch') will then be required to be validated in its entirety, transaction by transaction.

6.7.1 Synchronicity Failures

In v2, dispatch and echo evidence applies to signed verified quorum rounds after quorum 0 prefill dispatch, and to repair or reconciliation paths that collect the same evidence. It does not apply to quorum 0 itself. Quorum 0 is a non-consensus availability push: a missed prefill contact may reduce local availability and trigger manifest repair, but it is not counted as a failed signed consensus message.

During a verified quorum round, variances in node hardware, network delay, or Byzantine behavior can still prevent a quorum from progressing beyond dispatch. For example, a node may lack the bandwidth to deliver dispatch bytes in time, a Byzantine node may withhold dispatch entirely, or a correct node may be delayed while finishing a prior quorum. Echo messages mitigate this by allowing peers to sign what they have observed. Conceptually, each validator tracks a $q \times q$ evidence relation: one axis identifies the member expected to dispatch, and the other identifies the peer expected to echo what it observed. A validator ignores its own self-axis and accepts only signed evidence from distinct current validators.

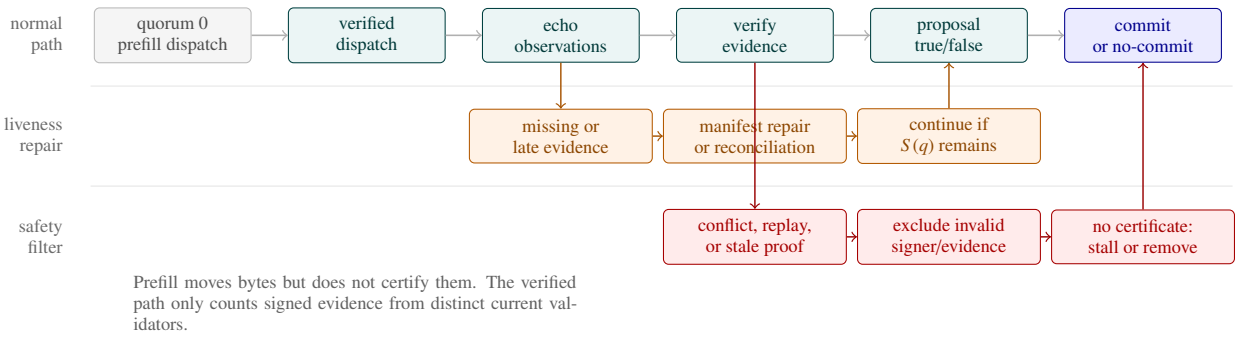
Figure 6 shows the stage order and the two fault branches. In a healthy verified quorum, all non-self dispatch and echo evidence is present, so the quorum can proceed through verification, proposal, and commit. If one correct member is slow or temporarily offline, the condition is first treated as a liveness and repair event: if the remaining distinct validators still meet the supermajority threshold $S(q)$, the quorum may certify the shared result and repair missing bytes or summaries separately. If the member is Byzantine, the same missing or inconsistent evidence is treated as a safety event. Conflicting, replayed, stale, or absent messages are excluded from the verified path, and the quorum commits only when the remaining evidence certifies one result; otherwise it stalls into repair or removal evidence.

6.7.2 Distribution Failures

In the case a quorum fails to reach consensus, any non-Byzantine nodes present within said quorum may miss block information that honest peers intended to share. The measurement for this case is termed failure bleed: the number of non-Byzantine nodes that lose state as a consequence of their association with Byzantine, unavailable, or partitioned peers. Earlier versions mitigated this by running a secondary consensus-tree recursion with appraisal and request messages. In v2, this is no longer the default epoch path. Deterministic prefill dispatch first places each local block with the peers that will carry it through later tensor dimensions, and signed reconciliation becomes the exception path when a certified summary quorum is unavailable. Reconciliation collects signed block manifests, reconstructs the candidate block set, and admits the result only when the same supermajority threshold used by the verified path is satisfied. This keeps the common case to one prefill hop plus the remaining verified rounds, while preserving an explicit repair path for unavailable summaries, dropped nodes, and future-round assists.

6.8 Transaction Validation

Following the completion of *block propagation* each node is expected to maintain a unified dataset, a constituent of all blocks distributed by every non-Byzantine node on the network. This information substantiates the *preliminary epoch*,



Prefill moves bytes but does not certify them. The verified path only counts signed evidence from distinct current validators.

Figure 6

Verified quorum stage flow. V2 begins with quorum 0 prefill dispatch as a non-consensus availability step, then proceeds left-to-right through signed dispatch, echo, verification, proposal, and commit. Slow or offline members enter the liveness-repair lane and may be bypassed when the remaining validators still satisfy $S(q)$. Byzantine, replayed, stale, or conflicting evidence enters the safety-filter lane and cannot certify a commit.

which must now be verified independently by each node to reach a finalized state. This finalized state is represented by an 'epoch hash' which simply confirms the unique and newly ratified condition of the ledger, verified by each node by use of a cryptographic signature. Despite maintaining an identical dataset, the protocol requires each validator to verify their *preliminary epoch* independently, as the consensus proof is not based on the work of each node, but the shared outcome of all nodes (i.e. the supermajority agreement of all independently achieved epoch states). If a method was implemented which returned an identical consensus proof, while not requiring each node to verify the entirety of each preliminary epoch, but rather a minority partition of the preliminary epoch, our protocol could achieve additional performance increases. Given our assumption regarding *state determinism*, as long as all nodes are provided an identical data input, we can expect all non-Byzantine nodes to concurrently reach an identical state. If some supermajority of nodes reached such a shared state, the epoch in question would be considered provably correct, and would pass validation.

The process to reach a finalized state requires each node to verify the legitimacy of every block and every transaction within the *preliminary epoch*. The process for verifying blocks and transactions is slightly different; however, the positive outcome of these procedures allows each node to affirm the state of their finalized epoch. We have discussed the behavior required to validate transactions and blocks in previous sections (referred to as *pre-validation*), but given the finality of this new procedure, the outcome of this process will impact the local ledger of each node, thus requiring additional novel validation proofs.

Prior to discussing the specific behaviors we implement during verification, it is valuable to summarize the data structure of the *preliminary epoch*, as it relates to the process of *transaction validation*. The goal of *block propagation* is to allow for the blocks formed in parallel by each node to be efficiently distributed, to inform every other node of all network appendages. However, given the requirement for concurrent computation across decentralized actors, we guarantee computation determinism by implementing a B-tree data structure during the local storage of blocks. This allows each block to be identically sequenced across actors, assuming equivalent input data was provided following *block propagation*. This uniform sequence of blocks helps induce *state determinism*, given no other faults occur during the verification process. By removing any uncertainty derived from a lack of concurrency among actors, disagreement among nodes is increasingly unlikely, and we can expect Byzantine nodes to occur at intervals equivalent to our assumption regarding *Byzantine faults*, or at other rates due to some uncertain malicious action.

The formal process we implement to reach an 'epoch hash' requires nodes to 'verify' the legitimacy of each transaction in a non-ordered manner (i.e. verifying transaction signatures, epoch nonce and hash, etc…), followed by the sequential verification of transaction requests to alter the ledger. We have chosen to isolate signature verification, as well as other credibility proofs, due to their relatively high compute cost and necessity for transaction approval. Once the 'credibility' information of a transaction has been verified, the content of said transaction may be verified to update the ledger.

To clarify, when we mentioned 'updating the ledger' we are referencing the updating of some key-value pointer, rather than the altering of some mutable dataset. Due to the 'append only' nature of the ledger, each transaction is stored in a time-sensitive manner relative to its predecessors. For this reason, the ledger is not one, but two data structures; the first is a time series data structure where each transaction points to its predecessor; the second is a simple hashmap

with the value of each index updated given the outcome of each transaction.

The process of verifying a transaction's credentials, otherwise known as the transaction header information, is referenced as occurring in a 'non-ordered' fashion as the verification of this information is not required to occur in some deterministic order across the entire dataset. The only information required to be processed in a distinct order is the ledger appended information, otherwise known as the transaction body, with each transaction reliant on the outcome of the prior transaction.

The specific serialization process we implement to reach a verified state requires each node to iterate over their stored B-tree in sequential order. This process is efficiently completed with the use of a B+Tree, which is optimized to allow for direct transposition between leaf nodes' memory. Starting from the first leaf in the dataset, with the block hash operating as the key of the leaf, the full block associated with that key is referenced. Similar to each transaction, a block maintains a header and body, with the header containing credential information while the body maintains novel transactions. The header of each block should have been previously verified during *block propagation*; however, if additional redundancy is required it is possible to verify the header of each block again. With a block verified as correct, the transactions stored within each block may now be verified in sequential order.

Given the block content is affirmed following reverse verification, the *transaction map* may now be iterated over to check for correctness. Assuming the header information of each transaction was previously approved, it would be the responsibility of the validator to sample the body information of each transaction sequentially, to check whether the information endorsed as part of the transaction is feasible, given the present state of the ledger. A transaction may fail at this stage given the wallet constituent to the transaction does not possess the necessary tokens, or if any other information related to the transaction is incorrect. If the transaction passes all stages of validation, the validator appends their ledger to accommodate the newly approved transaction.

It is important we rigorously follow the sequence of transactions and blocks when updating the ledger for two important reasons. The first is that some determinism is required to allow for future auditors to reverse engineer the *transaction validation*, to verify the state of the present epoch; the second is that due to the parallel nature in which blocks are formed it is possible that two independent transactions, depending on the order those transactions are read, may or may not conflict. An example of this behavior would be: if an individual begins the epoch with a starting balance of 100 units, and during the epoch, they post two transactions. In the first transaction, the individual receives 50 units, and in the second transaction, the individual sends 120 units. While from the temporal perspective of the individual, they were able to send 120 units as the net outcome of the two transactions would leave them with 30 units. However, from the perspective of the network, there is a 50% likelihood that the transaction sending 120 units will occur before the transaction receiving 50 units in the eyes of the protocol. If such an order occurs the network would see the transaction sending 120 units as invalid as the user would have a negative balance in their account, and as a result, the transaction would be invalidated. Due to the likelihood of such scenarios, it is important transactions are processed in a consistent and deterministic manner.

As each block in a given epoch was created in parallel, we have not yet discussed the methods implemented to generate an 'epoch hash,' which accounts for the ordering and verification of said blocks and their transactions. While one may assume we implemented the previously discussed concatenation procedure, due to the ability for invalidated transactions to be passed within a given block, we must introduce a method for handling this ambiguity. Following *block propagation*, if for any reason a transaction is invalidated, the ledger will not be appended with said transaction query; however, despite failing to be approved, we do not remove the transaction from the block, nor do we remove the entire block from the epoch. Instead, to resolve this issue we implemented a novel procedure we call 'transaction tagging.' *Transaction tagging* requires some number of additional bytes to be appended to the front of every hash. This preceding byte string is intended to be unique to each transaction type (signaling to any validator and future auditor how to handle the transaction) while displaying the verification status of the transaction.

Assuming a single bit is utilized to declare the verification status of each transaction (e.g. boolean of `true` ⟨1⟩ or `false` ⟨0⟩), when implemented with a single preceding byte, *transaction tagging* allows for the classification of 128 (2^7) unique transactions types. If it is in the interest of our platform to allow for a greater number of transaction variations, we could instead prepend each hash with two (2) bytes, for a total of 32,768 (2^15) unique transaction types. When observed within the historic record, as a transaction can be simply determined `true` or `false` — with `true` transactions valid following all verification steps, while `false` transactions are minimally valid during block formation but invalidated by a later procedure — we maintain the integrity of the original block, while clarifying the status of the transaction.

The specialized procedure we implement to reach an 'epoch hash' and 'updated ledger state' is similar to the procedure described in *Block Formation* [§6.6], with the exceptions that invalidated transactions are ignored, and the 'core' hash of existing data types (transactions and blocks) are not updated. Here, the 'core hash' is the portion after the *transaction tag*, excluding the mutable verification-status prefix.

It is important we allow for the non-core (the 'verification status') part of each transaction to remain mutable, as that operation is at the discretion of each validating node. Given the header of all transactions and blocks stored in the *preliminary epoch* have been validated, the node is then required to repeat the concatenation procedure, this time utilizing the entirety of each data type's hash; however, instead of updating the stored hash of each data type with the outcome of this procedure, we store this mutable 'derivative hash' in a separate data structure. Our reasons for not updating each data type with a new hash and accompanying signature are due to the high computational overhead for doing so, as well as the superfluous nature of the operation. Instead, similar to the formulation of a 'block hash', each 'epoch hash' is computed by passively iterating over the sequence of the epoch's embedded components (i.e. blocks and transactions). While iterating over this sequence prior to forming a finalized 'epoch hash' this temporary data structure is described as the 'derivative hash.' However, once the iterative computation is complete the resulting 'derivative hash' is equivalent to the 'epoch hash.' This process is displayed in *Figure 7*.

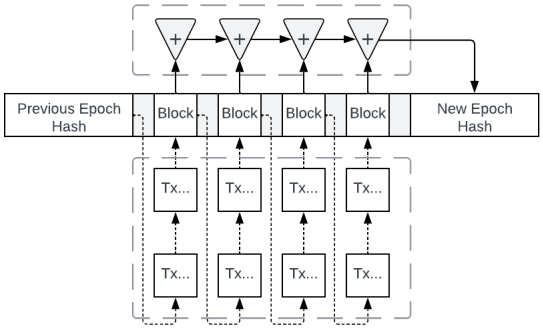


Figure 7

The procedure of generating a novel epoch hash ('New Epoch Hash') from the hash of each sequential transaction ('Tx...') and block ('Block'), concatenated ('+') with a previous hash.

Following the computation of a novel epoch, if one were interested in querying a specific historic transaction maintained by the ledger, there are two possible methods that could be implemented. The first option would allow for the querying actor to reference the most recent state of some index (e.g. account) on the ledger, and then iterate across all sequential transactions where an update was made to that index. This behavior is similar to that of a skip list [17]. The second option would allow the actor to use the location information of the transaction (i.e. the epoch hash or nonce, the block hash, and transaction hash) to query the transaction. Given each transaction maintains the nonce of its epoch, but not the hash of its epoch and block, an actor could also iterate over every transaction block.

7 Implementation Alignment

This v2.0.0 pre-release is intended to align the paper model with the protocol that is now implemented in the repository. The original protocol description remains a historical conceptual source, while this section is normative for current v2 claims where the two diverge. In particular, the thresholds, prefill-dispatch fanout, fair ordering, and reconciliation language below supersede older text that described the pre-2025 two-recursion model. The version boundary in §2 defines this section as the current v2 protocol claim, with separate trusted and trustless deployment boundaries, not merely an optimization appendix.

7.1 Implemented Quorum Parameters

The current protocol implementation defines a fixed quorum size of six in `src/algorithm.rs`. The same module defines the implemented supermajority threshold for a quorum of size q:

$$S(q) = q - \left\lfloor \frac{q}{3} \right\rfloor.$$

For a six-member quorum this gives $S(6) = 4$. The verified path's Byzantine safety proof uses

$$F(q) = \left\lfloor \frac{q-1}{3} \right\rfloor$$

as the maximum Byzantine population that can be tolerated inside that quorum while guaranteeing honest overlap between two accepted supermajority proofs. Therefore a six-member quorum can make liveness progress without two slow or absent members, but the honest-overlap safety proof admits one Byzantine signer in that six-member quorum. This distinction is important: slow-node tolerance and Byzantine equivocator tolerance are related, but not identical. Later proofs write $S(|C|)$ and $F(|C|)$ only when the threshold domain C is not necessarily a single quorum.

The implemented quorum schedule is deterministic. Nodes are sorted by public key, optionally shuffled from the epoch seed, and then assigned into a q-ary tensor coordinate system. For active validator count n and quorum size q, the schedule dimension is

$$D(n,q) = \max\left(1, \left\lceil \log_q(n) \right\rceil\right).$$

The ideal tensor capacity is $q^{D(n,q)}$. Non-ideal network sizes are mapped into the same deterministic schedule, with overflow validators appended into existing quorums according to the public ordering. This preserves a common quorum view across honest validators while allowing the implementation to run when n is not exactly a power of q.

Conceptually, a validator occupies a coordinate

$$a_i = (a_{i,0}, a_{i,1}, \ldots, a_{i,D-1}) \in [q]^D.$$

Each consensus layer moves across one tensor dimension. The local quorum for a layer consists of validators that share the node's other coordinates and vary along the active dimension. Earlier protocol descriptions used this schedule directly: for $n = 36$ and $q = 6$, the network formed a 6×6 tensor and ran a row-like round followed by a column-like round. If the first data-spreading round failed, however, a correct local block could remain trapped at the leaf of the tree and be lost from that branch.

The 2025–2026 v2 propagation model therefore adds deterministic *prefill dispatch*. Before verified consensus begins, each node sends its local block to the future contacts that will carry that block through the remaining tensor layers. This path is implemented and benchmarked in `src/subset_gossip.rs`; the deployable node runtime exposes the same contact selection through `src/algorithm.rs` and `src/runtime.rs`. When the opt-in TCP consensus driver is enabled for a multi-round topology, it emits the local prefill dispatch once, records that local availability evidence, and starts the verified stage schedule at the remaining tensor rounds. In the default unique coordinate-line profile, the remote prefill fanout is

$$P_{\text{view}}(n, q) = \min\left(n - 1,\ D(n, q)(q - 1)\right),$$

or

$$H_{\text{view}}(n, q) = 1 + P_{\text{view}}(n, q)$$

holders including the source node. The prefill stage replaces only the first data-spreading consensus round; the later verified rounds still run and certify the canonical epoch state. With $q = 6$ and $D = 10$, this means a node sends to 50 remote contacts, or 51 holders including itself, even though the tensor capacity is $6^{10} = 60{,}466{,}176$ validators.

The implementation also models a trustless withholding profile. Let ρ be the route width per future frontier. The default profile uses $\rho = 1$. When the simulator is configured with ϕ Byzantine full-payload withholders per branch, the route width becomes $\rho = \phi + 1$, and the planned remote fanout becomes

$$P_\rho(n, q) = \min\left(n - 1,\ \rho D(n, q)(q - 1)\right).$$

This keeps the single prefill hop, but increases initial availability replication so one configured withholding branch cannot orphan the block.

7.2 Quorum 0 Availability Boundary

We call this pre-consensus push *quorum 0* because it happens before the first verified quorum round, but it is not a quorum certificate. A quorum 0 delivery is only availability evidence: it stores a candidate block, its source, and its commitments. It does not admit transactions, finalize a block, advance membership, or change the validator set. Finality still requires the later verified dispatch, echo, verification, proposal, and commit sequence.

Let $R_e(u)$ be the deterministic quorum 0 recipient set for source validator u in epoch e. Honest nodes compute $R_e(u)$ from the current validator set V_e, the epoch seed, u's identity, q, and $D(n, q)$. Therefore a receiver can reject prefill from an unknown sender, a stale epoch, a reused nonce, or a sender-recipient pair that is not in $R_e(u)$. The trustless path then binds the delivered bytes to the signed block header and signature-tree commitment before those bytes can participate in a verified quorum transcript.

This boundary preserves the security claims despite the stage being non-consensus:

Byzantine behavior A Byzantine source may withhold, equivocate, replay an old payload, or send malformed bytes during quorum 0. These actions can reduce availability or waste receiver work, but they cannot make invalid state final. Later verified rounds count only distinct current validator identities and validate block commitments before proposal or commit evidence is accepted.

Fault tolerance Crash, slow, or partitioned recipients may miss prefill bytes. The route-width parameter $\rho = \phi + 1$ gives at least one live holder per modeled branch when at most ϕ holders withhold. If availability is still insufficient, reconciliation and repair handle the missing block explicitly; the block is not silently finalized.

Sybil tolerance Quorum 0 does not create validator identities. Recipient sets and holder evidence are derived from V_e, and duplicate keys, outsider keys, or identity-shifted replay evidence do not increase vote counts. Public registration and reconnection remain separate supermajority admission procedures.

Consistency Prefill moves bytes and commitments, not decisions. Once the same verified transcript and feature profile are committed, honest nodes still derive the same canonical epoch state. Conflicting quorum 0 payloads remain candidates unless a later verified certificate admits exactly one canonical view.

7.3 Trusted and Trustless Paths

The implementation now distinguishes two v2 execution paths. In the code these are exposed as `TrustMode::Trusted` and `TrustMode::Verified`; this paper uses *trustless* and *verified* interchangeably when describing the Byzantine-secure path.

Trusted path assumes a private known-operator cluster. It uses the same deterministic topology and block-integrity checks, but treats membership as the authentication boundary. Runtime messages can be emitted with default signatures, dispatch bodies can be scanned through the hot trusted path, and availability fetches and deliveries can use trusted wrappers. This is the lowest-latency path, but it is not a public Byzantine-security claim.

Trustless/verified path uses the full proof-bearing message sequence: dispatch, echo, verification, proposal, and commit. Votes are counted by distinct current validator identities, every accepted message is bound to the epoch, nonce, round, message kind, sender identity, and body being admitted, and dispatch payloads verify both block integrity and signature-tree evidence.

The epoch benchmark binary models these paths separately. For a two-layer $q = 6$ topology, the legacy trusted path has two data-moving stages, while the legacy verified path has ten proof-bearing network stages. With v2 prefill dispatch enabled, the first data-spreading verified round is replaced by one availability hop, so the same two-layer topology uses one prefill hop plus one five-message verified round. The trusted path remains a fire-and-forget data path: prefill improves first-round availability and later routing locality, but it does not add the four proof stages that only exist in the trustless path. The deployable binary keeps this driver behind `-auto-consensus`, while setting the default auto-driver horizon to the v2 two-round path. This makes the default node safe for embedding applications that want to drive epochs themselves, and v2-aligned for operators who explicitly enable the built-in healthy-quorum driver.

7.4 Production Runtime Surface

The deployable v2 runtime now includes production-facing persistence and recovery primitives in addition to the in-memory protocol state used by the simulator. A node may persist committed runtime state to a `RuntimeSnapshotV1`. This snapshot stores the committed epoch chain, local reachability metadata, trust mode, runtime mode, block queue capacity, and membership-removal policy. It intentionally excludes transient quorum votes, speculative consensus state, and in-flight local blocks. Restarted nodes therefore resume from the last committed checkpoint or use bounded catch-up reads, rather than replaying unfinished local votes as if they were committed state.

Block payload durability is separated from committed epoch checkpoints. `DurableBlockStore` stores verified blocks by content hash and maintains a nonce index for operator repair and restart tooling. When `BLOSSOM_BLOCK_STORE` is configured, the TCP node serves `GetBlock(nonce)` and bounded `GetBlocksByHash` repair requests from this store. Blocks are written only after local submission or verified received/prefill/redispatch payload checks have passed; malformed or hash-mismatched payloads do not enter the durable store.

The public wire surface also exposes read-only committed-chain recovery. The `EpochChain` request returns the local committed chain for small/local inspection, while `EpochChainRange` returns bounded pages for long-running nodes. Operator catch-up tools should prefer the range request so recovery does not require unbounded frame sizes.

The runtime also exposes the first concrete v2 recovery message surfaces. Future-round consensus messages are buffered until the referenced round becomes current, except for non-empty prefill-seeded future verification that can already be checked locally. Signed round-skip vote and certificate messages bind the epoch, nonce, source round, target round, and data-dissemination manifest hash. A certificate advances the local runtime round only after it passes current-validator supermajority checks and the manifest proves Byzantine-safe carried-block availability. `EchoRequest` can now return a targeted `EchoReDispatch` from locally verified or durable block availability, and manifest repair can fetch carried blocks by hash from eligible replica holders before retrying assist. `EchoReDispatch` and `ReconcileResponse` insert recovered blocks into local verified availability only after hash, nonce, epoch, and trust-mode validation. The `ReconcileAppraisal`, `ReconcileRequest`, `ReconcileResponse`, and `ReconcileCommit` messages establish the protocol envelope for rebuilt-state recovery. A `ReconcileCommit` can commit a locally verified rebuilt block set only after a distinct current-validator supermajority signs the rebuilt block-set hash. Speculative descendant replay remains recovery-driver policy rather than an implicit side-effect of the message.

The reconnect path is now represented in runtime code rather than only in the simulator. A reconnecting validator still submits a signed `NodeAdmission` for the next epoch target, but the admission is staged locally only after distinct active validators sign `ReconnectVote` records. Each vote binds the voter key, candidate key, admission hash, catch-up epoch hash, and catch-up nonce. Duplicate votes, stale checkpoints, non-validator votes, Sybil identity changes, and mismatched admissions do not contribute to the reconnect supermajority. This keeps reconnect as a block-carried membership transition while making the catch-up and active-validator approval gate explicit.

Discovery remains deliberately narrow. JSON bootstrap service files and bootstrap address-book sync seed reachability metadata for known peers, but they do not change verifier membership. Membership changes still require committed admission evidence. A global discovery network, deployment-specific rate limiting, key custody,

backup/restore drills, monitoring thresholds, and external security review remain operator responsibilities, not consensus shortcuts.

7.5 Hot-Path Runtime Optimizations

The current v2 codebase also includes performance work that is independent of the prefill round reduction. A local history audit shows a sequence of hot-path changes before the fair-ordering branch: zero-copy block handles and shared block indexes, cached signing state, streamed protocol hashes, deferred local-block hashing until close, avoidance of accepted-dispatch block clones, cached hot wire frames, overlay broadcast, trusted dispatch scanning, and trusted transaction paths. These changes reduce local memory movement, repeated serialization, repeated hashing, and avoidable signature work inside the runtime path.

These optimizations have different trust boundaries. Zero-copy handles, deferred hashing, cached frames, and shared indexes are mode-neutral: they improve the implementation without changing the proof requirement. Trusted dispatch scanning and unsigned trusted wrappers are mode-specific: they are valid only when membership is the deployment's authentication boundary. The trustless path still performs signed message verification, signature-tree validation, distinct-validator threshold checks, and authenticated availability transfer. Therefore the v2 performance story is not only "one fewer round." It is the combination of a shorter verified stage list after prefill dispatch and a lower-cost node hot path for the work that remains.

7.6 Propagation Policy Optimizations

The repository separates consensus safety from propagation-policy tuning in `crates/blossom-propagation`. The policy layer has two primary strategies. `PushFullBlocks` optimizes latency by sending the full payload directly, skipping recipients that are already known to hold the block. It is a one-hop strategy and is always admissible under the trustless boundary because the receiver still validates the block and consensus proof. `InventoryThenMissing` optimizes bandwidth by first advertising available hashes and then routing only missing blocks from selected holders. It adds an extra request/response step, so it is useful when the saved transfer time is larger than the added round-trip latency.

This distinction is security critical. In trusted mode, a hash-only manifest can be sufficient because known membership is the trust boundary. In trustless mode, inventory-based propagation is admitted only when the manifest is holder-signed or quorum-certified, the redundancy plan leaves at least one live holder after the configured Byzantine withholders are removed, and the policy does not rely on a single unauthenticated holder during consensus. If those constraints fail, the implementation rejects the optimized policy rather than weakening consensus safety. The adaptive policy therefore changes *how* bytes move, not *which* data can be accepted.

7.7 Filtered Availability Transfer

The availability-gossip feature adds a second bandwidth optimization for applications whose block commands contain recipient-scoped payloads. A block can carry filtered transaction slots: public metadata, target commitments, and payload hashes move in the control path, while the full payload bytes are served later through filtered fetch and batch-delivery requests. Trusted mode uses trusted fetch and delivery wrappers. Trustless mode signs fetches and deliveries and verifies freshness, sender identity, payload hash, and target commitment before the data can satisfy a missing slot.

This optimization is not a semantic shortcut. Filtered transfer changes the data plane by avoiding unnecessary bytes for nodes that are not targets of a payload, while the command hash, availability evidence, and validation gate remain deterministic inputs to the epoch transcript. The subset-gossip benchmark records this distinction with separate full-wire, subset-wire, repair, duplicate-suppression, and target-payload-completion metrics.

7.8 Default Trustless Epoch Ordering

The default trustless build enables fair block ordering, exposed in code under the fair-block-ordering compile-time feature. This feature is consensus-affecting: it changes the domain-separated epoch block tree from raw block-hash ordering to fair-order commitments. The active profile is labeled sha256+fair-block-ordering and is also encoded with a stable numeric feature code. Honest validators include the same feature-code byte sequence in the protocol hash namespace, so mixed raw-order and fair-order fleets reject one another before attempting to finalize an epoch.

The hash implementation also contains an `insecure-fast-hash` feature for private experiments and non-consensus benchmarking. That feature switches the protocol hash namespace to an `xxh3-128x2` profile and is intentionally excluded from the default trustless profile. A deployment cannot silently mix SHA-256 validators and fast-hash validators because the active feature-code registry is part of the protocol hash namespace.

Fair ordering is applied after blocks and transactions have entered the accepted set. It does not allow a node to make invalid data valid, and it does not change the quorum threshold. Instead, it derives an epoch-local ordering key from the canonical block transcript and the total transaction population. This prevents ledger-style applications from relying on raw hash order as a front-running surface. Explicit raw block-hash ordering remains available only for legacy or private compatibility builds that disable default features.

7.9 Runtime Reconciliation and Membership

The repository now contains runtime membership primitives and simulator-backed repair and reconnect behavior. Public nodes can submit a signed admission through `RegisterService`; that evidence is staged into a local block and only changes the verifier set if a supermajority of current verifiers commits the same admission at the epoch boundary. Dropped nodes do not re-enter merely by becoming reachable in the simulator model. A reconnecting node must ping active peers, obtain a catch-up proof for the current canonical state, and receive an admission supermajority. In trustless mode, these ping, catch-up, and admission thresholds are clamped to the same supermajority rule used elsewhere in the verified path. Duplicate votes, stale proofs, replayed evidence, and identity changes are counted as telemetry but do not increase the admission count.

If summary repair cannot find a certified supermajority for an epoch hash, the simulator falls back to signed block-set reconciliation. The reconciliation path rebuilds the candidate epoch from signed manifests and requires supermajority agreement before admitting the result as canonical. These behaviors are implemented in `crates/blossom-sim/src/epoch.rs` and are covered by deterministic scenario tests.

7.10 Bounded Awaits and Round-Skip Evidence

The v2 implementation separates a slow validator from a Byzantine validator. A node may observe future-round progress and assist that future round, but only through evidence bound to the current epoch, nonce, source round, target round, and certified data manifest. The protocol helper in `src/round_skip.rs` defines signed round-skip votes and certificates that count only distinct current-validator identities under the same supermajority rule used by the verified path.

The first fanout round is a data-availability boundary. If that round is skipped before a node's local block has entered the dissemination tree, the local block is dropped from the assisted future branch. If a later round is skipped after first fanout, the block is not dropped by default; it is carried forward by the certified manifest and remains eligible for repair and reconciliation. The manifest records per-block replica holders, so repair can reconstruct the full canonical data set from distributed replicas without requiring one peer to already hold the entire set immediately after first fanout. Data-bearing future-round votes require validation of the future body and parent data chain before they can contribute to advancement.

7.11 Telemetry and Observation

The v2 implementation embeds protocol telemetry and exports spans to the observer service in `crates/blossom-observer`. The observer is outside the measured node hot path, but the emitting cost inside each node is part of the runtime overhead. The current benchmark target is to preserve the telemetry overhead as an explicit protocol cost while keeping observer analysis separate from node finality measurements.

7.12 Benchmark Interpretation

The benchmark results should be read as implementation-aligned models, not as proofs that a deployment's network interface can sustain the required load. For $n = 36$, $q = 6$, and maximum-size blocks under the current frame limit, the model sends roughly the all-to-all lower-bound amount of full payload data. The remaining performance question is therefore dominated by the number of network stages, the supermajority latency ceiling, per-node bandwidth, and the cost of local verification.

8 Proof Obligations and Code Invariants

The protocol statements above are useful only if they can be traced to the same code paths that validators execute. This section records the proof obligations used by the v2 implementation. The claims are intentionally scoped: they prove deterministic agreement, threshold safety, bounded availability, and feature compatibility under the stated assumptions, while the benchmark section measures the cost of exercising those guarantees.

Let V_e be the current validator set for epoch e, let $n = |V_e|$, and let q be the quorum size. The implementation fixes $q = 6$ in `src/algorithm.rs` for the current production profile. Let π_e be the canonical validator ordering after optional epoch-seeded shuffle, and let $Q_i(r)$ be the quorum returned for validator i in round r.

8.1 Quorum Schedule Agreement

The quorum scheduler is implemented by `select_quorums`, `select_quorums_from_index_tree`, `deterministic_shuffle`, and `algorithm` in `src/algorithm.rs`. For fixed inputs (V_e, i, h_e, s), where h_e is the epoch seed and s is the shuffle flag, every honest node computes the same ordered index vector:

$$\pi_e = \begin{cases} \mathrm{sort}(V_e), & s = 0, \\ \mathrm{shuffle}_{h_e}(\mathrm{sort}(V_e)), & s = 1. \end{cases}$$

All later quorum arithmetic uses only π_e, q, n, and the local index of i. Therefore the mapping $i, r \mapsto Q_i(r)$ is a pure function of public epoch inputs. Two honest validators that share the same validator set and epoch seed cannot compute different quorums for the same member and round unless they disagree on one of those public inputs.

For ideal tensor sizes $n = q^D$, a node index can be written as

$$a_i = \sum_{j=0}^{D-1} a_{i,j} q^j, \qquad a_{i,j} \in [0, q-1].$$

Round r varies coordinate $a_{i,r}$ while holding the remaining coordinates fixed:

$$Q_i(r) = \left\{ \sum_{j \neq r} a_{i,j} q^j + c q^r \mid c \in [0, q-1] \right\}.$$

Thus $i \in Q_i(r)$ by choosing $c = a_{i,r}$, and if $j \in Q_i(r)$, then $Q_j(r) = Q_i(r)$. The implementation's non-ideal path preserves the same deterministic common-view property by mapping overflow validators into a deterministic schedule, appending overflow candidates, then sorting and deduplicating the result. The executable witnesses are `quorum_selection_is_deterministic_and_self_consistent_across_sizes` and `quorum_members_agree_on_their_round_group`.

8.2 Supermajority Safety

Throughout this paper, n denotes the active validator count $|V_e|$. Threshold arguments sometimes apply to the whole validator set and sometimes to a quorum-local domain. Quorum-local arguments use q for the configured quorum size and $q_Q = |Q|$ for the realized size of quorum Q. For the generic intersection proof, let $C \subseteq V_e$ be the current threshold domain. The verified path accepts a proof over that domain only when distinct current validators reach

$$S(|C|) = |C| - \left\lfloor \frac{|C|}{3} \right\rfloor.$$

The maximum Byzantine population covered by the honest-overlap proof is

$$F(|C|) = \left\lfloor \frac{|C| - 1}{3} \right\rfloor.$$

For any two accepted vote sets $A, B \subseteq C$ with $|A| \geq S(|C|)$ and $|B| \geq S(|C|)$, inclusion-exclusion gives

$$|A \cap B| \geq |A| + |B| - |C| \geq 2S(|C|) - |C|.$$

Writing $|C| = 3k$, $3k+1$, or $3k+2$ gives:

$\lvert C\rvert$	$S(\lvert C\rvert)$	$2S(\lvert C\rvert) - \lvert C\rvert$	$F(\lvert C\rvert)$
$3k$	$2k$	k	$k-1$
$3k+1$	$2k+1$	$k+1$	k
$3k+2$	$2k+2$	$k+2$	k

In all cases $2S(|C|) - |C| > F(|C|)$. Therefore two conflicting accepted supermajorities must share at least one honest validator whenever the Byzantine population is at most $F(|C|)$. For the implemented six-member quorum $q = 6$,

$$S(6) = 4, \qquad F(6) = 1, \qquad |A \cap B| \geq 2.$$

This is why the paper distinguishes slow-node tolerance from Byzantine safety: $\lfloor q/3 \rfloor$ validators in the quorum can be absent without stopping liveness, but the honest-overlap safety proof for $q = 6$ admits one Byzantine equivocator. The corresponding code surfaces are `supermajority_count`, `byzantine_fault_bound`, `max_liveness_omissions`, `min_supermajority_intersection`, and `supermajority_has_honest_overlap`. The same boundary is checked by unit tests and by the bounded Kani harness in `src/algorithm.rs`.

8.3 Distinct Identity and Admission Evidence

Thresholds are meaningful only if votes are counted by identity. Blossom therefore counts the set

$$E^\star = E \cap V_e$$

after duplicate removal. In this whole-validator case the threshold domain is V_e, so $|C| = n$, and the proof accepts only when $|E^\star| \geq S(|V_e|)$. The helpers `distinct_current_validator_count` and `has_distinct_supermajority` implement exactly this set operation, while `has_supermajority` rejects counts larger than n. A duplicated vote, an outsider vote, or a vote replayed under the same validator key cannot increase $|E^\star|$.

The register matrix in `src/register.rs` adds a second admission constraint for quorum-local message flow. Let Q_{reg} be the register matrix quorum and let $q_{\text{reg}} = |Q_{\text{reg}}|$. A node status can become `NodePassed` only when both the row and column axes reach $S(q_{\text{reg}})$, and an actual dispatch has been observed on that axis:

$$\begin{aligned} \text{pass}(v) = {} & \text{row}_v \geq S(q_{\text{reg}}) \\ & \wedge\ \text{column}_v \geq S(q_{\text{reg}}) \\ & \wedge\ \text{dispatchSeen}_v. \end{aligned}$$

Unknown senders and receivers are ignored before they can mutate the matrix. Consequently echo-only evidence can make a node pending, but it cannot pass a node that never supplied a dispatch. Public node registration and reconnection inherit the same rule: a candidate is staged as block evidence first, and the validator set changes only after current validators commit a supermajority of that evidence. For reconnects, the runtime adds a pre-staging gate: `ReconnectVote` records must be signed by distinct current validators and must bind the candidate key, admission hash, catch-up epoch hash, and catch-up nonce. Duplicate votes, outsider votes, stale checkpoints, and identity-shifted evidence are rejected before they can satisfy the admission threshold.

Future-round and recovery messages follow the same admission rule. A future-round message may be buffered, but it does not advance local state until the round is current or until a signed `RoundSkipCertificate` validates against

the current epoch, nonce, source round, target round, and `DataDisseminationManifest` hash. The manifest must prove that every carried block has the configured replica threshold among current validators. Thus a certificate can move the runtime's attention to a later round, but it cannot certify private round-zero data, unauthenticated block bytes, or a manifest that lacks Byzantine-safe availability evidence. Reconciliation messages are similarly non-canonical until a rebuilt block-set hash is backed by distinct current-validator evidence and then consumed by the normal verification/proposal/commit finality path.

8.4 Prefill Dispatch Availability

The v2 prefill model is implemented in `src/subset_gossip.rs`. Let

$$D(n, q) = \max(1, \lceil \log_q n \rceil)$$

be the tensor depth used by the prefill topology, and let ρ be the route width per future frontier. The prefill fanout is

$$P_\rho(n, q) = \min\left(n - 1,\ \rho D(n, q)(q - 1)\right).$$

Each tensor dimension contributes $q - 1$ remote contacts because the source node appears in every coordinate-line quorum and is counted only once. For $\rho = 1$, the local block therefore has

$$1 + D(n, q)(q - 1)$$

holders including the source node. For larger ρ, the planner selects ρ deterministic holders for each branch-equivalent contact class. Thus a source block has one owner plus a deterministic holder set covering the future contacts it will meet. The construction is a push path, not a fallback pull path.

If a branch can contain at most ϕ full-payload withholders and $\rho = \phi + 1$, then each future frontier contains at least one non-withholding holder under that branch assumption. The availability proof is simple: with $\phi + 1$ holders and at most ϕ withholders, pigeonhole leaves at least one holder able to push or prove the block. Prefill dispatch does not certify final state by itself; it only makes the owner block available before the verified path skips the legacy first data-spreading round. Later consensus, manifest checks, and reconciliation still certify the accepted epoch state.

The non-consensus nature of quorum 0 is therefore a safety boundary rather than a shortcut around consensus. For source u, receiver v, epoch e, and nonce η, a prefill delivery is modeled as

$$p_{u \to v} = (e, \eta, u, v, H(B_u), B_u).$$

The receiver may store $p_{u \to v}$ only when

$$\begin{aligned} \text{store}(p_{u \to v}) \Rightarrow \quad & u, v \in V_e, \\ & v \in R_e(u), \\ & H(B_u) = h_u, \end{aligned}$$

where h_u is the block commitment signed by u. This predicate is intentionally weaker than commit acceptance. It is a candidate-store rule, not a state-transition rule. A block can enter the canonical epoch set only after the later verified transcript satisfies the ordinary supermajority predicate over distinct current validators, or after a reconciliation certificate satisfies the same admission threshold for its manifest.

Consider the attack cases. A Byzantine prefill sender can equivocate by sending different B_u values to different recipients. If a verifier observes two distinct candidates for the same source, those candidates are keyed by the tuple (e, η, u), not merely by block hash. The local canonical verifier rule is:

$$\begin{aligned} \text{canon}(B_u) \Rightarrow\ & H(B_u) = h_u, \\ & B_u.\text{validator} = u, \\ & \nexists B'_u :\ H(B'_u) \neq H(B_u) \wedge B'_u.\text{validator} = u. \end{aligned}$$

Thus if a node observes validator u producing two distinct valid block hashes for the same epoch and nonce, u is marked as equivocating in that quorum state and all of u's observed candidate blocks are removed from the canonical verified set before proposal or commit can certify them.

If the equivocation is split so that each honest verifier initially sees only one candidate, the block-set hash still enters the ordinary verification threshold. A 3-3 split in a six-member quorum has no accepted verification hash because $S(6) = 4$. A 4-2 split can certify only the majority hash. Two conflicting block-set hashes cannot both certify under the Byzantine bound proved above, because two accepted verification sets must share an honest signer, and honest signers do not sign two different block-set hashes for the same (e, η, r). The attack can make the malicious validator lose its own block, split progress until reconciliation, or produce later equivocation evidence for removal, but it cannot introduce two canonical blocks for one validator into the same local canonical set and cannot fork honest state.

A replayed prefill from epoch $e' \neq e$ or nonce $\eta' \neq \eta$ fails the epoch/nonce binding before it can update the expected holder map. An outsider or Sybil key fails membership in V_e, and duplicate messages from the same validator count once because later certificates are sets of validator identities rather than message counts. A withholding sender can harm liveness or force repair, but it cannot certify a different block without crossing the same honest-overlap threshold proved above. These cases are covered by executable witnesses in `src/subset_gossip.rs`, `src/state.rs`, and `src/runtime.rs`, including stale epoch replay rejection, equivocated payload commitment rejection, branch-withholder survivability, and canonical exclusion of same-validator block equivocation.

8.5 Propagation Policy Admissibility

The optimized propagation crate is deliberately a policy layer rather than a consensus layer. Let σ be the selected propagation strategy and let H_b be the advertised holder set for block b. A direct full-block push is admissible in both trusted and trustless modes because acceptance still depends on the receiver's ordinary block, transcript, and proof validation. The strategy can waste bandwidth, but it cannot make an invalid block valid.

Inventory-based propagation has an additional proof obligation. A trustless node may request missing bytes from H_b only when the manifest is authenticated and the live-holder condition survives the configured Byzantine withholding bound. If $\Phi_b \subseteq H_b$ is the set of holders that may withhold the full payload, the policy requires

$$|H_b \setminus \Phi_b| \geq 1.$$

For the explicit branch-withholding model this is enforced by choosing holder redundancy larger than the configured withholding count. The implementation also rejects hash-only manifests in trustless mode and rejects single-holder pull plans when that holder is not authenticated by a signed or quorum-certified manifest. Therefore bandwidth optimizations can reduce duplicate transfer, but they cannot reduce the validator threshold, bypass message signatures, or substitute an unauthenticated pull for a verified consensus proof.

8.6 Round-Skip and Reconciliation

Round skipping is safe only when the skipped data boundary is explicit. A `RoundSkipVote` signs the tuple

$$\begin{aligned}\mathrm{msg}_{\mathrm{skip}} = (&\mathrm{voter}, \mathrm{lastEpoch}, \mathrm{nonce},\\ &\mathrm{fromRound}, \mathrm{toRound}, \mathrm{manifestHash}).\end{aligned}$$

A `RoundSkipCertificate` accepts only distinct current-validator votes that match that tuple, verify under the voter key, advance to a future round, and reach $S(|V_e|)$, because the round-skip certificate domain is the current validator set V_e. Stale epoch evidence and replayed nonces therefore fail before they can affect advancement.

The manifest binds the data decision. Its validation requires that sources and replica holders are current validators, that carried and dropped sets are disjoint, that replica evidence is present only for carried blocks, and that each carried block reaches the required replica threshold. Let r_b be the number of eligible replica holders for block b. For ordinary repair,

$$r_b \geq 1.$$

For a Byzantine-safe replica assumption with bound F, the code requires

$$r_b \geq F + 1,$$

so not every listed holder can be Byzantine. If first fanout has not completed and a local block has no carried replica, the helper must either drop that local block from the assisted future branch or serve the data before assisting. After first fanout, the block is carried by manifest evidence rather than dropped by default.

The runtime transition from prefill to the first verified round is not a finality transition. It only advances local scheduler attention so messages for the first verified tensor dimension are no longer buffered as future-round traffic. Canonical state still changes only after the verified dispatch, verification, proposal, and commit gates accept the same block set.

For reconciliation, `ReconcileCommit` is accepted only if the local node already holds a verified rebuilt block set whose hash equals the signed commit body and at least $S(|V_e|)$ distinct current validators sign that hash. A minority cannot use reconciliation to introduce unverified bytes, because the commit proof is checked against local verified availability before `advance_epoch` is called.

8.7 Fair Block Ordering and Feature Compatibility

Fair ordering is enabled by the `fair-block-ordering` feature and encoded in the protocol profile as `sha256+ fair-block-ordering`. Let $\mathcal{B}$ be the accepted block map and let

$$N_{\mathrm{tx}}(\mathcal{B}) = \sum_{b \in \mathcal{B}} |\mathrm{txs}(b)|$$

be the accepted transaction population. The implementation derives an epoch-local seed by hashing a domain separator, $N_{\mathrm{tx}}(\mathcal{B})$, the block count, and the accepted block transcript with byte positions reduced modulo $\max(N_{\mathrm{tx}}(\mathcal{B}), 1)$. Each block then receives an ordering key

$$\begin{aligned}K_b = H(&\mathrm{domain}, \mathrm{seed}, N_{\mathrm{tx}}(\mathcal{B}), h_b, H(b),\\ &\mathrm{validator}(b), |\mathrm{txs}(b)|).\end{aligned}$$

Sorting by K_b, with the raw block hash as a deterministic tie breaker, produces a total order for all honest nodes that accepted the same block map and feature profile. The feature registry in `src/hash.rs` rejects reserved feature identifiers, duplicate codes, unknown active codes, and unsorted profiles. Mixed raw-order and fair-order fleets therefore disagree at the protocol hash namespace rather than silently building different epoch trees.

This proof does not claim economic impossibility of every ordering attack. It proves the narrower consensus property: after the accepted block set is fixed, all compatible honest validators compute the same fair order, and a trader cannot choose final placement merely by grinding for a lower raw block hash.

8.8 Guarantee Boundary

The proofs above assume authenticated messages, deterministic serialization, the same current-validator set, the

same epoch seed, and at most $F(q_Q)$ Byzantine validators inside a verified quorum Q, where $q_Q = |Q|$, for the honest-overlap argument. They do not protect an application that inserts nondeterministic state into a block body, a deployment that mixes incompatible feature profiles, or a network whose Byzantine population exceeds the stated threshold. They also separate proof from measurement: finality latency, throughput, and bandwidth are not derived from the safety proof, but from the benchmark matrix and the observed supermajority order statistic used by the simulators.

9 Implementation-Aligned Projections

This section replaces the first-order projection model with one that matches the implemented protocol. The v1 model used a single latency value ℓ and treated it as a representative network latency. The implementation requires a more precise model: a quorum stage can advance once the required supermajority has arrived, so finality is governed by the slowest member of the fastest safe supermajority, not by arithmetic average latency.

9.1 Protocol Throughput and Finality

We use the following variables:

n active validator count.

q quorum size. The current implementation uses $q = 6$.

$D(n, q)$ tensor dimension selected by the implementation, where $D(n, q) = \max(1, \lceil \log_q(n) \rceil)$.

$R_2(n, q)$ active verified consensus rounds after v2 prefill dispatch. For the large-network path where $D(n, q) > 1$, $R_2(n, q) = D(n, q) - 1$. Without prefill dispatch, the legacy value is $D(n, q)$.

ρ prefill route width per future tensor frontier. The default single-route profile uses $\rho = 1$. In the trustless withholding model, $\rho = \phi + 1$, where ϕ is the configured number of Byzantine full-payload withholders tolerated per branch.

σ propagation strategy. The implementation models latency-oriented full-block push and bandwidth-oriented inventory-then-missing propagation separately.

$P_\rho(n, q)$ remote prefill recipient count under route width ρ. In the default single-route profile, $P_\rho(n, q) = \min(n - 1, D(n, q)(q - 1))$. In the general profile,

$$P_\rho(n, q) = \min(n - 1, \rho D(n, q)(q - 1)).$$

$H_{\text{view}}(n, q)$ unique coordinate-line view available to one validator after deterministic v2 prefill contacts:

$$H_{\text{view}}(n, q) = 1 + \min(n - 1, D(n, q)(q - 1)).$$

$q_Q = |Q|$ realized size of quorum Q. For ideal schedules $q_Q = q$; overflow quorums may be larger.

$S(q_Q)$ implemented quorum supermajority threshold, where $S(q_Q) = q_Q - \lfloor q_Q/3 \rfloor$.

$F(q_Q)$ Byzantine safety bound for quorum Q, where $F(q_Q) = \lfloor (q_Q - 1)/3 \rfloor$.

β transactions per block.

τ transaction payload size in bytes.

E block utilization, $0 \le E \le 1$.

ℓ one-hop latency in the uniform-latency reduction of the latency-distribution model.

$b_{1/3}$ bandwidth of the lower-third infrastructure class, measured in bytes per second.

$t_{1/3}$ thread count of the lower-third infrastructure class.

$\nu_{1/3}$ signature verifications per second per thread for the lower-third infrastructure class.

9.2 Quorum Scaling and Prefill View

The useful scaling property of the tensor schedule is not that a node directly talks to every validator. Instead, each node needs to touch the coordinate-line contacts that span the tensor dimensions used by the epoch. For an ideal tensor of depth $D(n, q) = \lceil \log_q n \rceil$, a node belongs to one quorum line in each dimension. Each line contributes $q - 1$ remote peers because the source node is shared by every line. Therefore the number of unique nodes in the local view is:

$$H_{\text{view}}(n, q) = 1 + D(n, q)(q - 1),$$

capped at n for small or non-ideal networks. This is the quantity a reader should use when asking how many direct contacts let one validator obtain a network-wide tensor view through the structured schedule. It is distinct from consensus finality: prefill provides availability and routing evidence, while later verified stages still provide signed commit evidence.

For $q = 6$ and $D = 10$, the tensor capacity is $6^{10} = 60{,}466{,}176$ validators, but the coordinate-line view for a single validator is $1 + 10(6 - 1) = 51$ nodes. Figure 8 compares this logarithmic view growth with a full P2P mesh and a centralized client/server architecture.

The implemented finality model must also include a latency distribution. Let $\delta^{(a)}_{u,v}$ be the one-way latency from sender u to receiver v during stage a. For a quorum Q with realized size $q_Q = |Q|$, define the stage latency ceiling for receiver v as the $S(q_Q)$-th order statistic of the valid arrivals:

$$\lambda_a(Q, v) = \text{os}_{S(q_Q)}\left(\left\{\delta^{(a)}_{u,v} : u \in Q\right\}\right).$$

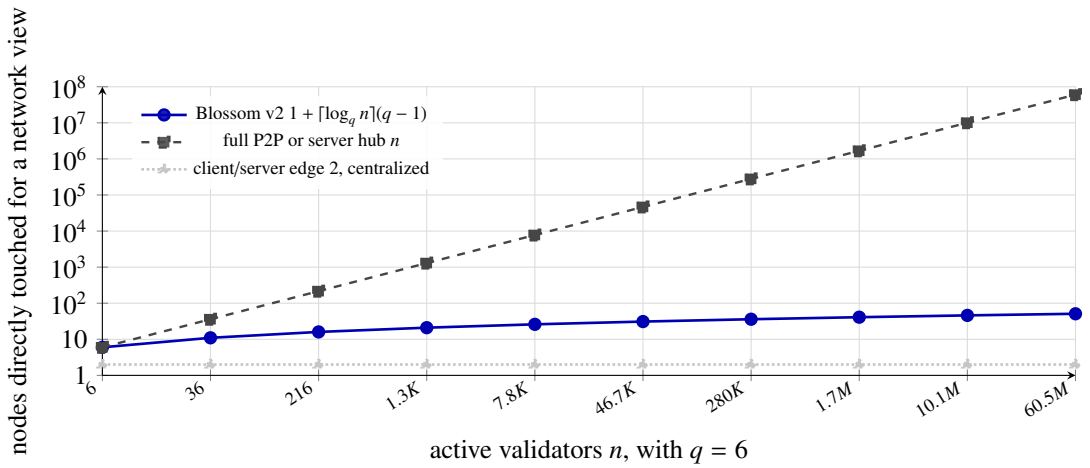


Figure 8

Scaling comparison for the number of nodes directly touched to obtain a full network view. A full P2P mesh gives each validator direct visibility over n nodes by maintaining $n - 1$ peer contacts plus itself, while a client/server design gives an edge node a constant path but concentrates the n-sized view at the hub. Blossom v2 avoids the hub while growing by tensor dimension: for quorum size q, a validator's direct coordinate-line view is $1 + \lceil \log_q n \rceil (q-1)$ nodes including itself.

The local node's own vote may be represented with latency 0. With $q = 6$, the threshold is $S(6) = 4$, so this expression waits for the fourth valid vote and excludes the two slowest or absent validators from the liveness ceiling.

The quorum stage latency is the slowest receiver that must participate in the accepted supermajority:

$$\Lambda_a(Q) = \mathrm{os}_{S(q_Q)}\left(\{\lambda_a(Q, v) : v \in Q\}\right).$$

This is the boundary between the fastest two-thirds and the slowest one-third of the quorum. It is intentionally not the mean latency. Average latency can be reported as an operational diagnostic, but it must not be used as the finality ceiling for a Byzantine-tolerant model.

Let A_μ be the ordered stage list for protocol mode μ. For the trusted path,

$$A_{\text{trusted}} = \{\text{dispatch}\}.$$

For the verified path,

$$A_{\text{verified}} = \{\text{dispatch}, \text{echo}, \text{verification}, \text{proposal}, \text{commit}\}.$$

The v2 prefill stage is a one-hop data-availability stage, not a consensus stage. Let $\Pi(n, q, \rho)$ be the modeled latency of that prefill hop. Larger ρ values increase initial bytes and the number of prefill recipients, but they do not add another consensus round. The modeled propagation latency for the prefill-dispatch path is therefore:

$$L_\mu(n, q) = \Pi(n, q, \rho) + \sum_{r=1}^{R_2(n,q)} \sum_{a \in A_\mu} \max_{Q \in \mathcal{Q}_r} \Lambda_a(Q).$$

For the legacy path without prefill dispatch, set $\Pi(n, q, \rho) = 0$ and replace $R_2(n, q)$ with $D(n, q)$.

If every directed edge has the same latency ℓ, and writing $D = D(n, q)$, this reduces to

$$\begin{aligned} L_{\text{trusted}}(n, q) &= (1 + |A_{\text{trusted}}|(D-1))\,\ell \\ &= D\ell, \end{aligned}$$

$$\begin{aligned} L_{\text{verified}}(n, q) &= (1 + |A_{\text{verified}}|(D-1))\,\ell \\ &= (1 + 5(D-1))\,\ell, \quad D > 1. \end{aligned}$$

The coefficient 5 is the verified message-stage count $|A_{\text{verified}}|$, not the quorum-derived value $q - 1$. The q-dependent term appears in the prefill fanout and local view formulas as $D(n, q)(q - 1)$. For $n = 36$ and $q = 6$, $D(n, q) = 2$. The trusted path still uses two data-moving stages, while the verified v2 path uses one prefill hop plus one five-message verified round, or six network stages. The legacy verified path without prefill uses ten network stages.

The block-time model is:

$$B_t = C_{\text{form}}(n) + \frac{W_\mu(n, q, \beta, E, \tau)}{b_{1/3}} + \frac{V(n, \beta, E)}{\nu_{1/3} t_{1/3}} + L_\mu(n, q),$$

where W_μ is implementation-modeled wire traffic and V is the number of required verification operations. Throughput is:

$$T_\mu = \frac{\beta n E}{B_t}.$$

The wire term $W_\mu(n, q, \beta, E, \tau)$ must be interpreted with the selected propagation strategy σ. Full-block push has lower stage count and higher duplicate payload traffic.

Inventory-then-missing has lower payload traffic when recipients already hold many blocks, but it pays an additional hash-advertise and missing-byte exchange. The adaptive implementation selects inventory only when the modeled saved transfer time exceeds the extra round-trip cost and the selected trust boundary admits the plan. In trusted mode this can use hash-only inventory. In trustless mode, the same projection must include manifest authentication, holder redundancy, and failure of unsafe single-holder pull plans; otherwise the projection is measuring a private optimization, not the public verified protocol.

For all-node convergence rather than finality, replace the order statistic in $\Lambda_a(Q)$ with a maximum over all non-faulty receivers. That stricter model is useful for catch-up and observer health analysis, but it is not the right ceiling for finality throughput when the protocol can finalize after a safe supermajority.

9.3 Latency Ceiling Proof Sketch

The implementation accepts a stage proof only after it observes $S(q_Q) = q_Q - \lfloor q_Q/3 \rfloor$ distinct valid signatures from quorum Q. Therefore, at most $\lfloor q_Q/3 \rfloor$ validators can be slow, absent, or non-responsive without preventing liveness. The earliest time at which a receiver can accept the stage is exactly the arrival time of the $S(q_Q)$-th valid message. This is an order statistic, not an average.

Safety follows from quorum intersection. Any two accepted vote sets of size $S(q_Q)$ intersect in at least $2S(q_Q) - q_Q$ validators. For $F(q_Q) = \lfloor (q_Q - 1)/3 \rfloor$, this intersection is always larger than $F(q_Q)$, so under the Byzantine bound the intersection contains at least one honest validator. Since an honest validator signs only one valid body for a given epoch, nonce, round, and stage, two conflicting accepted proofs cannot both be valid. Whole-validator admission proofs use the same formula with the explicit domain size $|V_e| = n$.

Thus, the correct latency ceiling for finality is the slowest arrival inside the first safe supermajority. The slowest one-third can affect catch-up time, observer health, and eventual all-node convergence, but it does not define the finality ceiling unless the protocol is configured to require every node to complete the stage.

9.4 Fault Tolerance

The implemented verified path requires supermajority evidence for epoch approval, repair, reconnect ping evidence, reconnect catch-up proofs, and reconnect admission votes. The threshold is always counted over distinct current validator identities and is bound to the current epoch and proof body. This prevents replayed evidence, duplicate votes, and Sybil identity changes from increasing the proof count.

The v2 safety claim is therefore narrower and more precise than the informal v1 statement: a six-member quorum can continue without two slow participants, but the honest-overlap Byzantine safety proof for conflicting supermajority proofs admits one Byzantine equivocator in that six-member quorum. Larger validator sets preserve the standard less-than-one-third Byzantine bound.

9.5 Failure Bleed

Failure bleed remains the case where a faulty quorum prevents honest members from learning data that the rest of the network has accepted. The implemented mitigation is now expressed as reconciliation rather than as a purely theoretical second recursion. When a node lacks a certified summary quorum for an epoch, it collects signed manifests, rebuilds the candidate block set, and admits the result only after the same supermajority threshold is satisfied. This behavior is exercised in the simulator and should be modeled as an exception path with additional latency and bandwidth, rather than as the default epoch path.

9.6 Dynamic Utilization

The dynamic utilization model continues to use E as the block utilization term, but the v2 model moves utilization through the implementation-modeled wire function $W_\mu(n, q, \beta, E, \tau)$. For $q = 6$ and $n = 36$, measured wire traffic is close to the all-to-all lower bound because every node must eventually learn every node's block. Smaller or non-square quorums can exceed that bound due to repeated full-payload propagation across intermediate quorum rounds.

The legacy maximum-load projection figures remain useful as historical context for the original model.

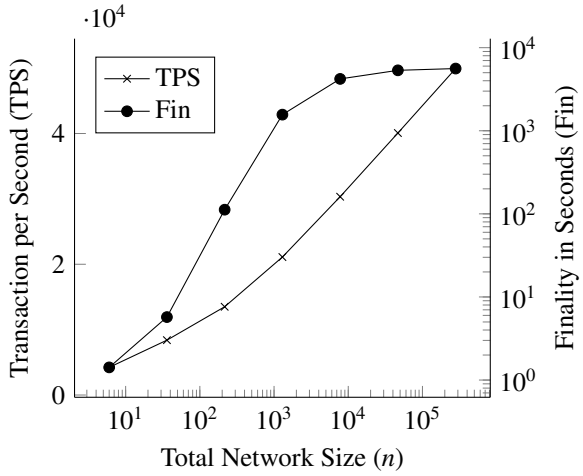


Figure 9

The network throughput and finality as the number of network members increases, assuming the network is under maximum possible load.

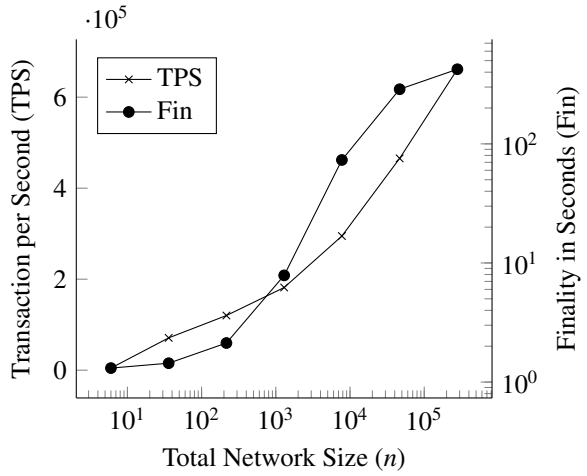


Figure 10

The network throughput and finality as the number of network members increases, assuming the network is under maximum possible load.

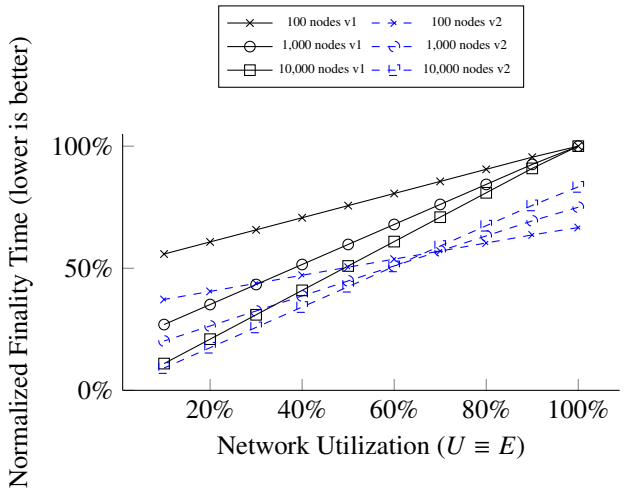


Figure 11

Projected v2 finality utilization envelope overlaid on the historical v1 utilization model. The y-axis is normalized finality time against v1 full-saturation finality for each network size, so lower percentages represent faster epoch finality. For $q = 6$, v2 scales finality by $r(n) = \max(1, \lceil \log_q n \rceil - 1)/\lceil \log_q n \rceil$, so the blue curves show the lower finality-time envelope available at the same network utilization.

The utilization overlays below retain the historical v1 utilization curves as the baseline and add v2 operating envelopes in blue. The finality envelope remains normalized against v1 full-block saturation, so the reader can see the absolute finality-time reduction available at the same block saturation; in this graph, lower values are better. The throughput-retention envelope is normalized against each protocol's own full-saturation throughput, so it is capped at 100% and shows the percentage change in throughput as utilization changes. The same projection term from the benchmark overlay is used here:

$$r(n) = \frac{\max(1, \lceil \log_q n \rceil - 1)}{\lceil \log_q n \rceil}.$$

Finality is scaled by $r(n)$. Absolute throughput is scaled by $1/r(n)$ in the benchmark overlay, while the utilization graph reports percentage retention against each protocol's own saturated throughput.

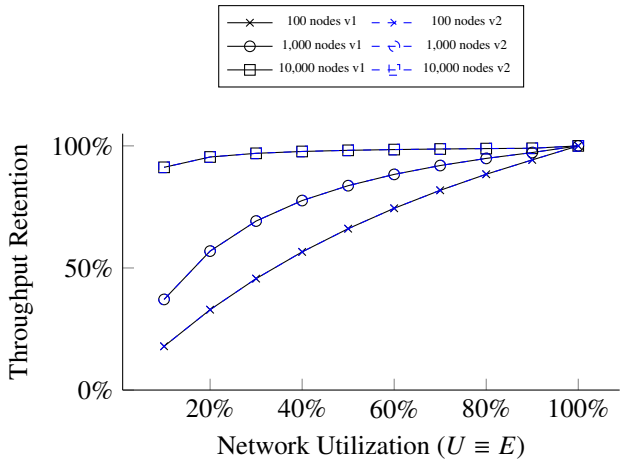


Figure 12

Projected v2 percentage change in throughput versus network utilization, overlaid on the historical v1 utilization model. Throughput retention is normalized against each protocol's own full-saturation throughput, so the curves are capped at 100%. *Under the current one-wave projection, v2 improves the absolute full-saturation TPS shown in Figure 17, while retaining the same bounded percentage response to lower utilization.*

10 Benchmarks

To demonstrate the feasibility of our protocol, as well as validate the performance estimates we have referenced in the preceding sections of this document, we have undertaken a series of experiments to quantify the variety of behaviors related to our protocol. These measurements have been used to classify the headline performance estimates previously referred to as throughput (i.e. TPS) and transaction finality (i.e. finality), as well as some less consequential discoveries of note.

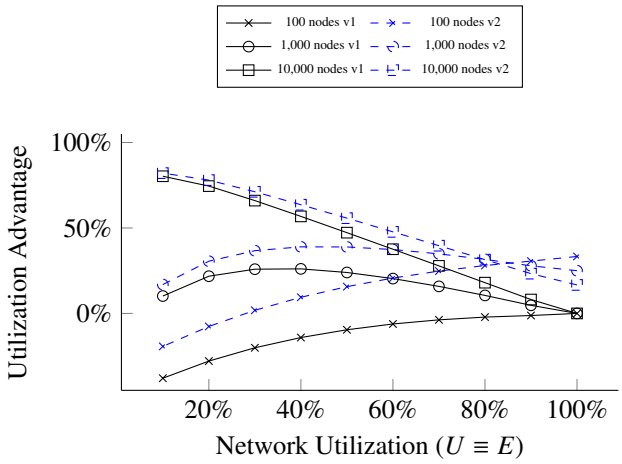


Figure 13

Projected v2 utilization-advantage overlay for selecting an operating network utilization. The plotted score is $A_{\text{util}}(E) = \mathcal{T}(E) - \mathcal{F}(E)$*, where* $\mathcal{T}(E)$ *is the capped throughput-retention curve from Figure 12 and* $\mathcal{F}(E)$ *is the finality envelope from Figure 11. Higher values indicate a better finality-throughput trade at that utilization under the projection.*

10.1 Methods

We would like to preface the coming disclosure of *Blossom* benchmarking with several statements clarifying the context of our experiments, as well as provide the reader with some reasonable expectations one must hold when analyzing the data. We believe these acknowledgments to be necessary, as there has been an observed behavior among industry members to purport performance benchmarking as “real world" performance, without much credence to the true circumstances with which the authors collected said measurements.

With that being said, we distinguish historical deployment measurements from current v2 validation runs. The original v1 figures were collected in the AWS cloud under sterile laboratory conditions. The v2 overlays, trusted/trustless comparisons, and fault-validation checks are generated by deterministic repository harnesses so they can be reproduced and audited with the code. When a statement is made, we notate the relevant infrastructure or model assumption as well as the observed hardware utilization when available. Unless otherwise stated, measurements in this section should be read as engineering evidence under controlled conditions, not a guarantee of future deployment behavior. For this reason, despite any specific performance figure we reference in this document, the primary claim is scalable performance: as membership grows, the communication shape grows by tensor dimension instead of requiring every validator to maintain an all-to-all view.

To efficiently scale our network simulation across environments, we deployed blossom nodes using a container architecture, implementing Kubernetes to manage the orchestration of the associated distributed infrastructure [4]. Each instance of the application was maintained as a replica service, with a permissioned address limiting communication to network members. Given nodes operate as replica devices without the need to add or remove peers to or from the network, following cluster deployment nodes were required to log with a centralized Redis database (DB) [5]. Nodes, expecting to maintain a ledger of n peers, would then query the DB until they had stored the credentials for the entire network. For the release of *Blossom* we recognize that such centralized infrastructure is non-viable, and intend on implementing a distributed hash table (DHT), such as Kademlia [9], or a similar service to facilitate the decentralized addition and removal of nodes. However, for the purposes of our testing, we do not expect these approaches to return different results by a statistically significant margin.

To manage the undertaking of our simulation, many of the variables we have discussed throughout this document (network size, quorum size, block size, signature scheme, etc) are passed to nodes using environment variables at deployment. This allows us to easily initialize network simulations of variable size while guaranteeing no resources are utilized for non-relevant nodes. With the network successfully initialized, we may then begin sending requests to the network via a client. These requests are formatted as regular `HTTP messages` and allow for nodes to `GET` some network information from a node, or `POST` new transactions to a node. To undergo our testing of transaction throughput and finality we focused on `POST` requests, deploying specialized clients with the ability to create simulated transactions for the network to process. To limit any bottlenecks associated with any single client, nodes are designed to handle parallel communication, thus we are able to deploy more than one (1) client per network node if necessary; however, across our testing, we implemented the standard ratio of one (1) client for every six (6) nodes.

To better understand the scalability of *Blossom* we have incorporated four (4) variables for our testing which may be strategically modulated to isolate the protocol’s unique limiting factors. Those variables include instance type (i.e. node `CPU` and `RAM`), node locations (i.e. colocated or globally distributed), signature verification, and network size. The implementation now exposes this distinction directly through trusted and verified runtime modes. Historical deployment figures in this section are therefore best read as private-cluster measurements unless they explicitly include the verified signature and proof path. Current simulator and benchmark binaries can run the same topology with `-trusted` disabled, v1 or v2 protocol selection, prefill-dispatch controls, hash-advertise controls, latency distribu-

tions, and Byzantine-withholding knobs; those runs report the extra metrics needed to compare private trusted throughput against public trustless safety.

10.2 Results

The network sizes we implemented as part of this experiment utilized the minimum quorum size q of Blossom, with the number of nodes increasing at an exponential rate from the base value of q ($n = q^{[1:4]}$). For deployment purposes, we have limited the largest number of nodes we test to $n = 1296$, with the variety of instances similarly limited, with a minimum resource allocation of about one (1) vCPU and maximum resource allocation of about eight (8) vCPU per node. For reference, the common threshold of decentralization for a network is 1,000 nodes.

The following figures (*Figure* 14, *Figure* 15, and *Figure* 16) represent the data collected when all network members (nodes) are flooded with a surplus number of transactions, with the total cumulative number of transactions processed by the network equaling more than five (5) million. Due to our unique network architecture, and effective block cap, each node is only able to distribute a maximum of 1,000 transactions each epoch, thus the number of transactions distributed each epoch is a function of the number of nodes:

$$N_{\text{epoch}} = 1000n.$$

Due to the nature of this test (over-saturating the network with transactions) we expect to see finality times increase with the number of active nodes, while (due to the need to complete compute-heavy tasks, such as that of signature verification, and our ability to process this task in a parallelized manner) we expect that as the number of CPU cores utilized by each node increase, the finality time of each epoch will decrease, thus increasing the overall throughput of the network. We expect these changes in performance to be linearly correlated with the change in compute resources (CPU) available to each node, up until some threshold is met.

10.3 Projected V2 Benchmark Overlay

The historical figures above are retained as measured v1 deployment results. To compare v2 against the same benchmark scenarios, we project the v2 protocol onto the original benchmark axes instead of introducing a separate simulation matrix. The projection uses the same quorum size as the measured benchmarks, $q = 6$, and sets $d = \lceil \log_q n \rceil$. V1 requires d dissemination waves across the quorum tensor. V2 prefill dispatch moves the first payload distribution step out of the verified consensus path, so the projected payload-ready finality is

$$Fin_{v2}(n) = Fin_{v1}(n)\frac{\max(1, d-1)}{d}.$$

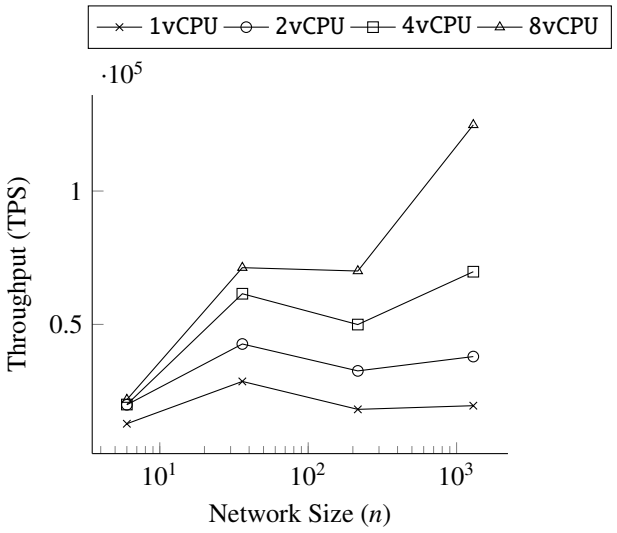


Figure 14

Average network throughput (TPS) at a given configuration (indicated above).

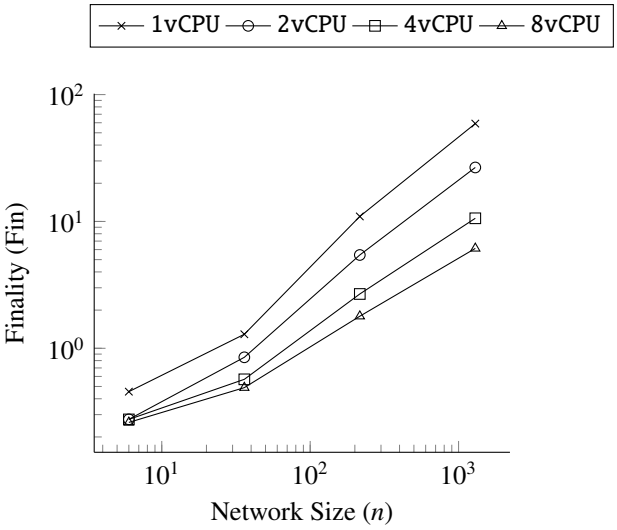


Figure 15

Average network finality at a given configuration (indicated above).

The $\max(1, d-1)$ term leaves the single-quorum $n = 6$ case unchanged, because there is no later consensus wave to remove. For $n > 6$, v2 removes one verified wave and keeps the remaining quorum tensor unchanged.

For this projection we hold transactions per epoch constant. V2 changes the time required to make each epoch payload-ready; it does not change the per-node block cap used in the original benchmark. Therefore projected v2 throughput is computed as the same accepted transaction vol-

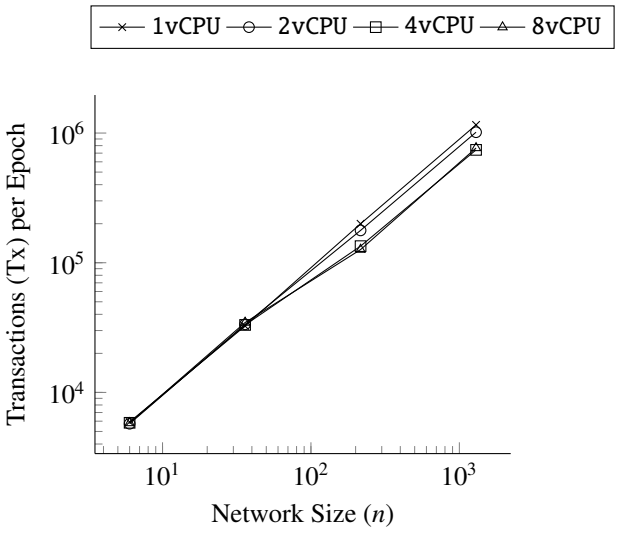


Figure 16

The average number of transactions in each epoch at a given configuration (indicated above), where each transaction is equivalent to 400 `bytes` *of data (0.4kB).*

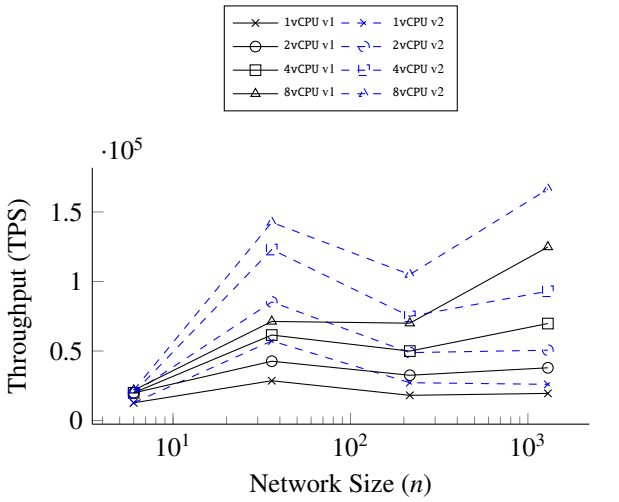


Figure 17

Projected v2 throughput overlaid on the historical v1 throughput benchmark. The projection holds the observed transactions per epoch constant and scales throughput by the v2 finality projection in Figure 18.

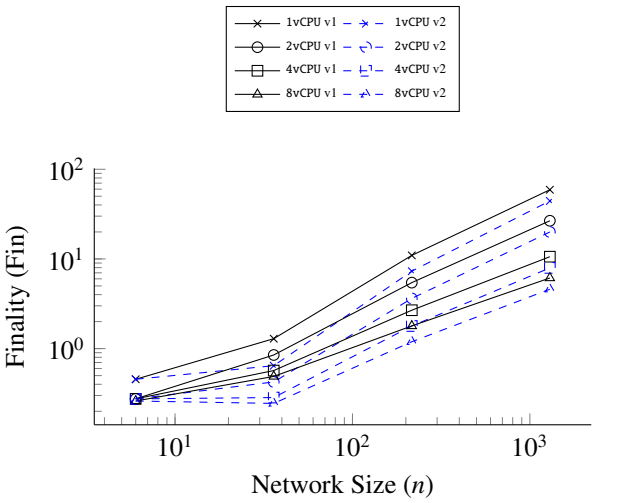


Figure 18

Projected v2 finality overlaid on the historical v1 finality benchmark. For network size n and quorum size $q = 6$, the projection uses $d = \lceil \log_q n \rceil$ and models v2 as removing one verified consensus wave from the payload-ready path when $d > 1$: $Fin_{v2} = Fin_{v1} \max(1, d-1)/d$.

ume divided by the projected finality. Figures 17, 18, and 19 copy the v1 benchmark charts and add the projected v2 curves in blue.

This means the benchmark comparison is intentionally asymmetric: v1 lines are historical measurements, while v2 lines isolate the protocol-path change described in §2.2. The overlays therefore answer a narrow question: how the same measured scenarios move when the first verified data-spreading wave is replaced by deterministic prefill dispatch, with the remaining benchmark assumptions held constant.

The projected v2 lines should not be read as hiding the trusted/trustless split. Trusted v2 removes signature-tree verification and proof-bearing message stages from the hot path, while trustless v2 keeps those checks and uses prefill to remove only the first data-spreading consensus wave. The subset-gossip harness also measures propagation-policy choices separately: full-block push, hash inventory, duplicate suppression, filtered payload repair, modeled finality latency, total wire bytes, and per-node bandwidth. A public deployment must use the trustless/verified policy constraints described in §7; a private deployment may choose the lower-latency trusted profile if operator trust is part of the application assumption.

10.4 Trusted versus Trustless V2 Runtime Cost

To isolate the cost of the deployment trust boundary, we ran the current `blossom-subset-gossip-bench` harness in v2 mode with the same quorum size, command count, command size, target count, and latency profile while only changing `-trusted`. The benchmark used $q = 6$, 64 commands per node, 1 KiB commands, three target payloads per command, and a 50 ms even one-hop latency. The 36-node and 216-node points are 100-epoch deterministic runs. The 1,296-node point is a paired single-epoch scale sample, included to show the same relationship at the historical q^4

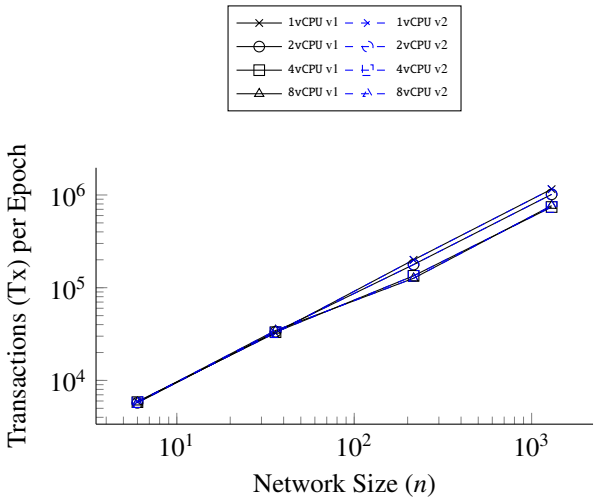


Figure 19

Projected v2 transactions per epoch overlaid on the historical v1 transaction benchmark. V2 changes dissemination latency and repair exposure, not the per-node block cap, so this projection keeps the accepted transaction count per epoch unchanged; the TPS gain in Figure 17 comes from lower finality.

benchmark size without making the document wait on a long soak run. The plotted summary rows are recorded in `paper/data/trusted-vs-trustless-v2-summary.csv`.

Figure 20 shows the expected mode split. The trusted path retains dispatch and prefill availability movement, but it does not pay the four proof-bearing stages after each dispatch stage. The trustless path keeps echo, verification, proposal, and commit evidence after prefill dispatch. Under the even-latency model this produces a roughly 5× modeled-finality difference for the post-prefill consensus rounds. Throughput does not scale by exactly the same factor because payload-ready TPS includes prefill latency, subset delivery, and modeled repair/data-plane timing.

10.5 Latency and Fault Validation

The public v2 claim is not only that prefill dispatch reduces a modeled round; it is also that the verified path remains correct when ordinary network cost and fault pressure are present. To check that boundary we ran a 36-node validation matrix at 100 ms one-hop latency. Clean trusted and trustless runs used 12 epochs with 20 ms jitter. Faulted trustless runs used 16 epochs with 50 ms jitter, dropped/fuzzed/spiked messages, repair attempts, dropped-node reconnect attempts, and a partition scenario. The committed summary is in `paper/data/latency-100ms-validation-summary.csv`; the raw run artifacts were produced under `target/validation-latency-100ms`.

Figure 21 shows the two conclusions that matter for release readiness. First, trusted and trustless deployments have different latency envelopes by construction: trusted mode treats membership as the authentication boundary, while trustless mode keeps the proof-bearing verified round after prefill. Second, the faulted trustless scenarios converged to one final epoch hash with all 36 nodes correct and no incorrectly lost local blocks. The partition case is intentionally severe: it dropped 26,220 messages and still finished with a single canonical view after repair and reconnect logic.

10.6 Discussion

The results from our testing, depicted in *Figure* 14, *Figure* 15, and *Figure* 16, are closely aligned with our expectations going into the experiment, with rates of change in performance between `vCPU` thresholds (e.g. `1vCPU` to `2vCPU` to `4vCPU` to `8vCPU`) hitting some threshold at lower network sizes (6 nodes and 36 nodes), while changing at a near-linear rate for larger network sizes (216 nodes and 1,296 nodes). We expect the observed *near-linear* rates of change, especially at larger network sizes, to derive from fluctuating epoch sizes, with *transactions per epoch* noticeably decreasing as the compute resources (`vCPU`) available to each node increase.

The reasons we believe to be the cause of the observed fluctuation in *transactions per epoch* are derived from the nature of the protocol. As finality decreases in correlation with the increased availability of `vCPUs` per node, each node is required to collect the same block of transactions (ideally 1,000 transactions). With less time to complete the same task, while prioritizing the validation of an increasingly large number of transactions each epoch, we expect the likelihood each node fails to properly receive, validate, or batch client transactions to increase by a reasonable margin of error. While we do not observe the loss of any data as a consequence of this transaction process, with the distributed ledger at the end of each experiment maintaining the expected transactions, the number of epochs processed by the networks was larger than the ideal expectation across all network sizes. This is to be expected; however, further analysis to understand the rate of change across network sizes would be valuable in our process to understand the change in network performance across scales.

10.7 Projection Modeling

When referencing the projections originally postulated in *Protocol Throughput and Finality* [§9.1], we found that our formulas were successful in their ability to project the performance of *Blossom* within a reasonable margin of error. However, this correlation is highly dependent on the use of appropriate variables. In the scenario where incorrect variables are implemented, projected performance may drasti-

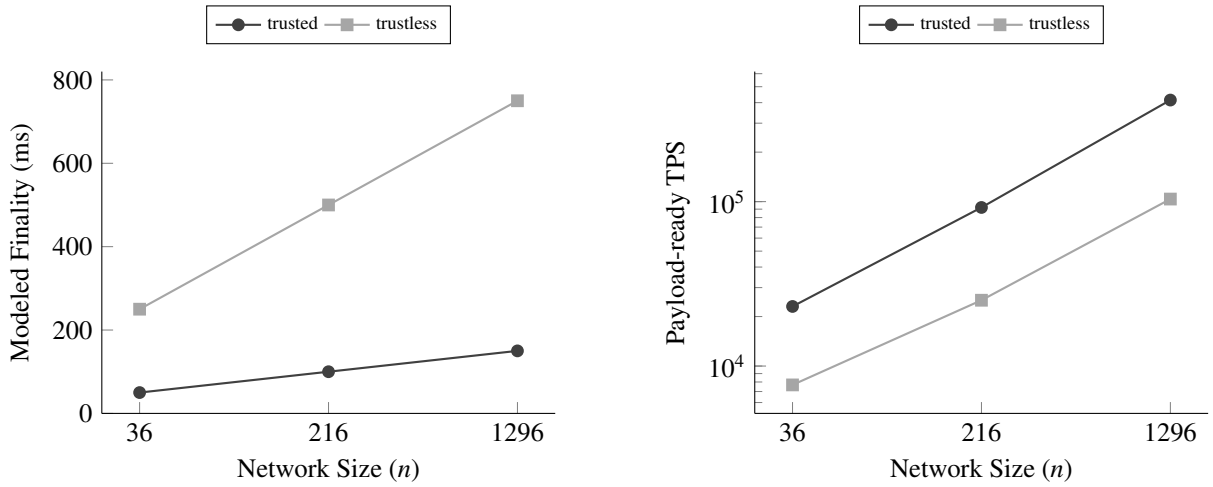


Figure 20

Head-to-head v2 trusted and trustless subset-gossip benchmark under the same topology, payload size, and 50 ms one-hop latency. The 36-node and 216-node points are 100-epoch deterministic runs with 64 commands per node and 1 KiB commands. The 1,296-node point is a paired single-epoch scale sample used to show the same path relationship at the historical q^4 benchmark size. Trusted mode removes proof-bearing control stages from the hot path; trustless mode retains dispatch, echo, verification, proposal, and commit evidence after prefill dispatch.

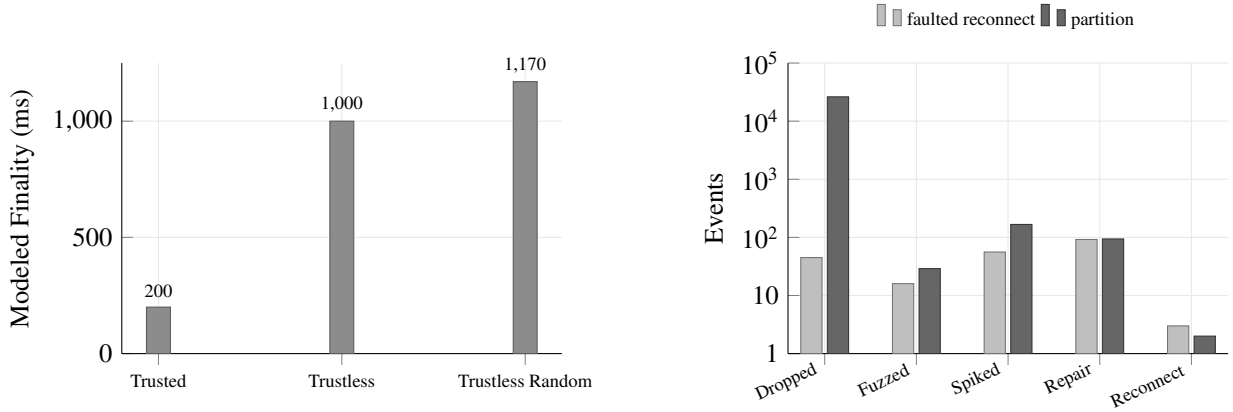


Figure 21

Validation under a 36-node, 100 ms latency profile. The left panel shows the modeled finality cost for trusted and trustless v2 epoch runs: trusted mode pays one modeled dispatch stage, while trustless mode pays the verified proof-bearing stages after prefill. The right panel summarizes the fault-pressure scenarios on a log scale. Both faulted runs still ended with 36 correct nodes, zero incorrectly lost local blocks, and one final epoch hash.

cally over-perform or under-perform the real-world outcome of the given node configuration. When referencing the testing completed in this section, the variables which accurately reflect the network configuration we implemented for our testing are $\ell = 335$ ms and $\nu = 34700$ signature verifications per second per thread. All other variables will be updated in line with the configuration of the network simulation.

When assuming a constant utilization rate (E) across network scales, we may project the optimal performance of the network given the same benchmarks outlined in the previous figures (*Figure* 14, *Figure* 15, and *Figure* 16). These projections are discussed in §9.6 and shown in *Figures* 9 and 10.

10.8 Protocol-Shape Comparison

External network TPS and finality claims change over time and are rarely measured under comparable validator counts, hardware, workload sizes, trust assumptions, or transaction semantics. For release purposes, the more sta-

ble comparison is therefore the communication shape: how many peers a validator must directly touch to obtain a complete network view, and where the bottleneck is placed.

Figure 8 compares those shapes. A full P2P mesh gives every validator direct visibility over n nodes, but each node pays $O(n)$ direct contact cost. A client/server design can make the edge path constant, but it concentrates the $O(n)$ view and failure domain at the hub. Blossom v2 keeps the view decentralized while growing by tensor dimension: for quorum size q, the direct coordinate-line view is

$$H_{\text{view}}(n, q) = 1 + \lceil \log_q n \rceil (q - 1)$$

holders including the source node. With $q = 6$, a network with more than 60 million validators has a 51-node direct view for one validator's prefill availability path. The benchmark implication is not that every deployment beats every external protocol on every workload. The reproducible claim is that Blossom's communication surface grows logarithmically in the validator count while preserving later verified quorum evidence in the trustless path.

11 Future Additions

In this section of the document, we discuss future additions to the Blossom protocol which we have not yet implemented. However, these additions may provide additional performance increases from those previously displayed. The additional functionalities we mention in this section are not guaranteed to be implemented or researched by our team. Rather, these implementations are highlighted due to their unique technologies, which when proven, may provide meaningful benefits to the Blossom protocol. The specific approaches are as follows:

11.1 Aggregated Signature Schemes

Aggregated signature schemes would allow for multiple signatures to be expressed as a single signature. Implemented in Blossom such an algorithm would allow all of the transactions stored within a single block to be validated by use of a single signature. This would drastically reduce the computational overhead of signature verification by several orders of magnitude (dependent on the number of transactions in a block). While this technology is promising, the algorithms are still in the research phase with other blockchain foundations working on implementing the technology within their protocols. However, despite the successful implementation of this novel signature scheme, there are serious security worries which may discredit this approach. To avoid speculation, we will avoid discussing the possible tradeoffs of this technology in detail; however, we would certify that we would not implement a novel signature scheme unless it was guaranteed to have a statistically impossible likelihood of being broken. In the meantime, we are focusing on research on advancing the efficiency of Blossom with existing signature schemes, and it is our belief that given any possible future innovation, the work we are doing in the present to extend the functionality of our protocol is a much more valuable endeavor than speculating on unproven technology.

11.2 Quantum Safe Cryptography

Quantum Safe (QS) crypto is an overarching classification for novel crypto algorithms which are intended to resist attacks from classical computers, as well as quantum computers. The National Institute of Standards and Technology (NIST) selected its first post-quantum cryptography (PQC) algorithms in 2022 and released its first three finalized PQC standards in 2024: FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) [2, 12]. These algorithms are now concrete implementation candidates rather than speculative future standards. For Blossom, the open question is performance and operational fit: post-quantum signatures can materially increase signature and bandwidth costs, so any trustless profile that adopts them must be benchmarked against the same finality, throughput, and verifier-cost model used for the current signature path. Given the pace of progress in the field, we expect post-quantum profiles to become increasingly practical, but we would treat them as explicit feature-code profiles rather than silent replacements for the current hash and signature namespace.

12 Conclusion

This paper describes a novel consensus protocol that implements cryptographic techniques and self-healing behaviors to provide deterministic transaction finality while allowing for network throughput to scale with the number of participating nodes. These improvements are a first in the space of decentralized computing and are achieved by the systematic recursive subsampling of network actors, resulting in the distribution of state information in logarithmic time. The implications of these technical improvements are broad, with the ability for the protocol to serve new sectors and technical capabilities in the short-term, while increasing the overall efficiency and adoption of the decentralized technology in the long term.

13 Notation Glossary

This glossary fixes the notation used by the v2 implementation-aligned sections. Historical v1 projection text keeps its local context, but current v2 claims use the meanings below. In particular, n always denotes the active validator count and q denotes quorum size. Generic threshold proofs write the counted domain size explicitly as $|C|$ so it is not confused with the quorum variable.

v1. Historical Blossom protocol baseline and measured deployment benchmark path. V1 uses one verified data-spreading consensus wave per tensor frontier and is retained for comparison.

v2. Current implementation-aligned Blossom protocol path. V2 adds deterministic prefill dispatch before verified consensus and treats the later verified rounds, manifests, reconciliation, membership evidence, and feature compatibility checks as the normative trustless behavior.

Trusted path. Private known-operator deployment mode exposed as `TrustMode::Trusted`. It preserves deterministic topology and block integrity checks, but treats membership as the authentication boundary and can use unsigned trusted dispatch and availability wrappers.

Trustless/verified path. Public Byzantine-security deployment mode exposed as `TrustMode::Verified`. It verifies signed messages, signed or quorum-certified evidence, distinct-validator thresholds, signature trees, and authenticated availability transfer.

13.1 Validators, Quorums, and Thresholds

e**.** Epoch index.

V_e**.** Current validator set for epoch e.

$n = |V_e|$**.** Active validator count. This is the size of the current network view, not a quorum-local count.

q**.** Configured quorum size. The current production profile uses $q = 6$.

$q_Q = |Q|$**.** Realized size of quorum Q. In ideal tensor schedules $q_Q = q$; overflow quorums may be larger.

i, j, u, v**.** Validator indices. In latency formulas, u is the sender and v is the receiver.

h_e**.** Epoch seed used by deterministic scheduling.

s**.** Shuffle flag in quorum schedule proofs. $s = 0$ means sorted validator order and $s = 1$ means epoch-seeded shuffle.

π_e**.** Canonical validator ordering for epoch e, after optional deterministic shuffle.

a_i**,** $a_{i,r}$**, and** c**.** Tensor coordinate for validator i, the coordinate value at round r, and a local coordinate value varied inside a quorum schedule proof.

$Q_i(r)$**.** The quorum containing validator i at round r.

Q**.** A generic quorum.

$\mathcal{Q}_r$**.** Set of quorums participating in round r.

C**.** Current threshold domain in a proof. It may be the whole validator set or a quorum-local set.

$|C|$**.** Size of the current threshold domain when the proof is not specifically quorum-local.

$S(q)$**,** $S(q_Q)$**, and** $S(|C|)$**.** Supermajority threshold. For the configured quorum size:

$$S(q) = q - \left\lfloor \frac{q}{3} \right\rfloor.$$

For a realized quorum use $S(q_Q)$; for a non-quorum threshold domain use $S(|C|)$.

$F(q)$**,** $F(q_Q)$**, and** $F(|C|)$**.** Byzantine safety bound. For the configured quorum size:

$$F(q) = \left\lfloor \frac{q-1}{3} \right\rfloor.$$

For a realized quorum use $F(q_Q)$; for a non-quorum threshold domain use $F(|C|)$.

Q_{reg}**,** q_{reg}**.** Register matrix quorum and its size, $q_{\text{reg}} = |Q_{\text{reg}}|$.

$E^\star$**.** De-duplicated admission evidence restricted to the current validator set.

M_{msg}**.** Per-peer message factor used in the historical consensus-tree cost comparison.

$\mathcal{D}_\alpha^i$**.** Dataset held by the α-th participant in illustrative quorum q_i.

Σ_i**.** Union of datasets accumulated by illustrative quorum q_i.

13.2 Topology and Prefill Dispatch

$D(n, q)$**.** Tensor depth selected by the implementation:

$$D(n, q) = \max(1, \lceil \log_q n \rceil).$$

$R_2(n, q)$**.** Active verified consensus rounds after v2 prefill dispatch. For $D(n, q) > 1$, $R_2(n, q) = D(n, q) - 1$.

ρ**.** Prefill route width per future tensor frontier.

ϕ**.** Configured number of Byzantine full-payload withholders tolerated per branch. The trustless withholding profile sets $\rho = \phi + 1$.

$P_\rho(n, q)$**.** Remote prefill recipient count:

$$P_\rho(n, q) = \min(n - 1, \rho D(n, q)(q - 1)).$$

$H_{\text{view}}(n, q)$**.** Unique coordinate-line view available to one validator after deterministic v2 prefill contacts:

$$H_{\text{view}}(n, q) = 1 + \min(n - 1, D(n, q)(q - 1)).$$

r_b**.** Number of eligible replica holders for block b in a reconciliation manifest.

σ**.** Propagation strategy. The current implementation models `PushFullBlocks` for low latency and `InventoryThenMissing` for bandwidth reduction.

H_b**.** Advertised holder set for block b in an inventory or manifest-based propagation plan.

Φ_b**.** Modeled withholding subset of H_b. Trustless inventory requires at least one live holder after removing Φ_b.

13.3 Latency, Throughput, and Utilization

μ**.** Protocol mode index, such as trusted or verified.
A_μ**.** Ordered stage list for mode μ.
a**.** Protocol stage index inside a stage list.
$\delta^{(a)}_{u,v}$**.** One-way latency from sender u to receiver v during stage a.
$\lambda_a(Q, v)$**.** Receiver-local stage latency ceiling: the $S(q_Q)$-th valid arrival at receiver v for quorum Q.
$\Lambda_a(Q)$**.** Quorum stage latency ceiling: the $S(q_Q)$-th receiver-local ceiling among quorum members.
$\Pi(n, q, \rho)$**.** Modeled latency of the one-hop prefill dispatch stage.
ℓ**.** Uniform one-hop latency used only in the simplified latency reduction.
β**.** Transactions per block.
τ**.** Transaction payload size in bytes.
E**.** Block utilization, with $0 \le E \le 1$.
U**.** Network utilization used in utilization charts. In the projection figures, $U \equiv E$.
$C_{\text{form}}(n)$**.** Modeled cost of forming a block for a network of size n.
$W_\mu(n, q, \beta, E, \tau)$**.** Implementation-modeled wire traffic in mode μ.
$V(n, \beta, E)$**.** Required verification operations.
$b_{1/3}$**.** Bandwidth of the lower-third infrastructure class, measured in bytes per second.
$t_{1/3}$**.** Thread count of the lower-third infrastructure class.
$\nu_{1/3}$**.** Signature verifications per second per thread for the lower-third infrastructure class. When no lower-third qualifier is needed, the paper uses ν.
B_t**.** Block time.
$L_\mu(n, q)$**.** Modeled propagation latency in mode μ.
T_μ**.** Throughput in mode μ.
$r(n)$**.** v2 projection scaling term used in benchmark overlays:

$$r(n) = \frac{\max(1, \lceil \log_q n \rceil - 1)}{\lceil \log_q n \rceil}.$$

$\mathcal{T}(E)$**.** Capped throughput-retention curve in the utilization overlay.
$\mathcal{F}(E)$**.** Finality envelope in the utilization overlay. This is intentionally distinct from $F(q)$, the Byzantine safety bound.
$A_{\text{util}}(E)$**.** Utilization advantage score:

$$A_{\text{util}}(E) = \mathcal{T}(E) - \mathcal{F}(E).$$

13.4 Fair Ordering

$\mathcal{B}$**.** Accepted block map for an epoch.
b**.** A block when used as a subscript or element of $\mathcal{B}$.
$N_{\text{tx}}(\mathcal{B})$**.** Accepted transaction population:

$$N_{\text{tx}}(\mathcal{B}) = \sum_{b \in \mathcal{B}} |\text{txs}(b)|.$$

h_b**.** Block hash or block-header hash used in fair ordering.
K_b**.** Fair ordering key assigned to block b.

13.5 Reserved Usage

The v2 proof sections do not use x as a network size or threshold-domain variable. Figure source may still use x as a plotting coordinate inside PGFPlots, but mathematical claims use n for active validators, q for configured quorum size, q_Q for realized quorum size, and $|C|$ for generic non-quorum threshold-domain size.

References

[1] Allavena, A., Demers, A., and Hopcroft, J. E. Correctness of a gossip based membership protocol. In *Proceedings of the twenty-fourth annual ACM symposium on Principles of distributed computing* (2005), pp. 292–301.

[2] Boutin, C. NIST announces first four quantum-resistant cryptographic algorithms | NIST. *NIST News* (July 2022).

[3] Buchman, E. Tendermint: Byzantine fault tolerance in the age of blockchains (doctoral dissertation), 2016.

[4] Burns, B., Beda, J., Hightower, K., and Evenson, L. *Kubernetes: up and running*. " O'Reilly Media, Inc.", 2022.

[5] Carlson, J. *Redis in action*. Simon and Schuster, 2013.

[6] Castro, M., and Liskov, B. Practical byzantine fault tolerance and proactive recovery. *ACM Trans. Comput. Syst. 20*, 4 (Nov. 2002), 398–461.

[7] Durstenfeld, R. Algorithm 235: Random permutation. *Commun. ACM 7*, 7 (July 1964), 420.

[8] Kwon, J., and Buchman, E. Cosmos whitepaper. *A Netw. Distrib. Ledgers* (2019), 27.

[9] Maymounkov, P., and Mazières, D. Kademlia: A peer-to-peer information system based on the XOR metric. In *Peer-to-Peer Systems*, Lecture notes in computer science. Springer Berlin Heidelberg, Berlin, Heidelberg, 2002, pp. 53–65.

[10] Miller, A., Xia, Y., Croman, K., Shi, E., and Song, D. The honey badger of BFT protocols. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, Oct. 2016), ACM.

[11] Nakamoto, S. Bitcoin: A peer-to-peer electronic cash system. *Decentralized Business Review* (2008).

[12] National Institute of Standards and Technology. NIST releases first 3 finalized post-quantum encryption standards. https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards, 2024. Released August 13, 2024; updated August 29, 2025.

[13] Oneil, P., Cheng, E., Gawlick, D., and Neil, E. The log-structured merge-tree (LSM-tree). *Acta Informatica 33*, 4 (1996), 351–385.

[14] Oracle. LinkedHashMap (java platform SE 8). https://docs.oracle.com/javase/8/docs/api/java/util/LinkedHashMap.html, Apr. 2023. Accessed: 2023-4-18.

[15] Prometheus. Prometheus - monitoring system & time series database. https://prometheus.io/. Accessed: 2023-4-18.

[16] Proof of stake instead of proof of work. https://bitcointalk.org/index.php?topic=27787.0. Accessed: 2023-4-18.

[17] Pugh, W. Skip lists: a probabilistic alternative to balanced trees. *Commun. ACM 33*, 6 (June 1990), 668–676.

[18] Rocket, T. Snowflake to avalanche: A novel metastable consensus protocol family for cryptocurrencies, 2018.

[19] Sekniqi, K., Laine, D., Buttolph, S., and Sirer, E. G. Avalanche platform. *Netw. Distrib. Ledgers* (2020).

[20] Strauch, C., Sites, U. L. S., and Kriha, W. NoSQL databases. *Lecture Notes, Stuttgart Media University 20*, 24 (2011).

[21] Yakovenko, A. Solana: A new architecture for a high performance blockchain v0. 8.13. *Whitepaper* (2018).